



Is this build failure related to my patch? An empirical study of unrelated build failures in continuous integration

Yonghui Andie Huang^{1,2} · Daniel Alencar da Costa¹ · Grant Dick¹ ·
Mariam El Mezouar³ · Liwen Xiao¹

Received: 8 February 2025 / Accepted: 24 April 2026 / Published online: 21 May 2026
© The Author(s) 2026, modified publication 2026

Abstract

In a hectic Continuous Integration (CI) environment, where several builds are triggered concurrently, legitimate build failures (e.g., not caused by flaky tests) may not always be related to the current push. These unrelated build failures can burden developers as they devote hours to attest whether errors are truly associated with their present changes. In this paper, we extract 77,354 CI build failures from 7 open source projects to understand and identify unrelated build failures. We attempt to provide an indication for developers about whether a build failure is likely to be related to the current push or not. Our results reveal that developers likely invest a median of 4 hours to determine whether a build failure is (un)related to their pushes. We perform a document analysis on a sample of 371 unrelated build failures (based on the 95% confidence level and 5% confidence interval from 10,316 potentially unrelated failures) to understand why build failures are deemed as unrelated by developers. The themes generated from our document analysis reveal that *unrelated tests failures* represent 20% of the cases of why build failures are deemed unrelated by developers. To predict whether a build failure is unrelated to the current push, we extract 33 features from issue reports, issue comments, and from the commits pertaining to the triggering push. We build semi-supervised PU-learning models over seven Apache projects and achieve precision ranging from 0.70 ± 0.01 to 0.88 ± 0.02 , recall ranging from 0.30 ± 0.03 to 1.00 ± 0.00 , and F1-scores ranging from 0.44 ± 0.03 to 0.91 ± 0.00 , while the area under the ROC curve (AUC) spans 0.63 ± 0.02 to 0.97 ± 0.03 . Our analysis of feature importance reveals that (i) the time taken from a submitted patch to the build-triggering push (CI latency), (ii) build failures sharing similar error messages with recent failures, and (iii) the number of comments preceding the build failure, are all efficient indicators for identifying potential unrelated build failures. The semi-supervised approach proposed in this work can help developers identify build failures that are unrelated to their current push, providing actionable guidance such as re-running builds, inspecting infrastructure logs, or prioritizing code-level debugging based on prediction outcomes.

Keywords Non-code-related failures · Continuous Integration (CI) · Empirical study · Issue resolving

Communicated by: Bram Adams.

Extended author information available on the last page of the article

1 Introduction

Continuous Integration (CI), a software development practice that involves the frequent integration of code changes, has gained traction in both industry and academia (Fowler and Foemmel 2006; Duvall et al. 2007; Shahin et al. 2017; Donca et al. 2022). Researchers have thoroughly investigated the potential benefits and costs of CI in the last years (Soares et al. 2022). For instance, CI has empirically shown associations with increased productivity (Bernardo et al. 2018; Vasilescu et al. 2015; Santos et al. 2022) despite potential challenges such as extra efforts required to address CI configuration issues (Zampetti et al. 2019).

According to Duvall et al. (2007), CI is a combination of several interrelated practices, including frequent check-ins (e.g., frequent *Git Pushes*), short build cycles, a comprehensive test suite, and proper build feedback (or *bells and whistles*). Regarding *proper build feedback*, it is important that development teams have visibility into when builds fail, who caused the failure, and the reason for failure. Indeed, Duvall et al. (2007) emphasize the utmost importance of fixing a build failure before moving on to other tasks. This is because developers may relax their adherence to CI practices if they perceive that builds are constantly failing. For example, developers may choose to forgo thorough checks of their changes before pushing to the mainline (i.e., the build is always failing anyway).

Notwithstanding the importance of fixing build failures, given the prevalence of distributed software development settings, where multiple pushes are made concurrently, it becomes increasingly challenging to fix build failures immediately. In fact, Ghaleb et al. (2019b) revealed a phenomenon called *cascading build breakages*, where a build fails not due to the changes of the current push, but due to changes inherited from a previous push that initially caused the build to fail. As such, an adverse effect of *cascading build breakages* is that developers often wonder whether their code changes are directly responsible for the build failure or if the failure is unrelated to their changes, possibly caused by another developer's push.

Much research has studied the challenges involved in adopting CI, including complexities around build notifications (Elazhary et al. 2021; Shahin et al. 2017; Zampetti et al. 2020). Elazhary et al. (2021) studied three companies using CI and reported that they face “notification fatigue” due to frequent builds, while also requiring discipline to monitor build status. Zampetti et al. (2020) studied CI practitioners to uncover bad patterns related to CI usage. Regarding the status of builds, the authors found that build notifications can often be verbose, containing unrelated information, while also containing the output of different build tasks, which can confuse developers when it comes to fixing build failures.

While previous research has highlighted issues related to sub-optimal build notifications, our work is the first to address the problem of automatically identifying unrelated build failures. We define an unrelated build failure as a failure that occurs during the continuous integration (CI) process but is not caused by the current code push. We do not define “unrelated failures” as failures that are entirely unrelated to the codebase or project as a whole. Rather, we refer to failures that are unrelated to the most recent code push. More specifically, an “**unrelated build failure**” as one where (i) no error messages, stack traces, or test failures can be traced back to the source files modified in the associated code push, (ii) the root cause lies outside the submitted code changes before the merging stage, or (iii) follow-up developer comments explicitly confirm that the issue is unrelated to the submitted code.

It is important to distinguish unrelated build failures from direct regression failures. A direct regression failure occurs when a developer's code change causes other tests to fail. In such cases, the root cause lies within the submitted code changes, and condition (ii) is not satisfied. Therefore, direct regression failures are classified as related, not unrelated. In contrast, in large-scale concurrent development environments, build failures often occur due to changes introduced by other developers. For the current developer, such failures are unrelated because the root cause lies outside their submitted code changes.

In practice, unrelated failures can lead to significant developer frustration and wasted effort, as teams may spend time investigating issues that are not actionable for their current change. If developers can be certain that a build failure is not related to their push, they can use this information to make decisions, such as proceeding with the integration or better debugging the build breakage. This is particularly crucial in distributed development settings, where different collaborators simultaneously contribute to the project, such as open-source projects or remotely distributed teams (Lanubile et al. 2010; Hertel et al. 2003).

It is important to distinguish unrelated build failures from flaky tests. *Flaky tests* (Luo et al. 2014) refer to automated tests that exhibit non-deterministic behavior, producing inconsistent results despite unchanged code. Unrelated build failures, on the other hand, pertain to legitimate failures in the build process caused by changes unrelated to the current code push. For example, configuration issues or unforeseen problems due to differences in environments, can contribute to build failures related to code pushes. As such, our goal in this study is to understand and potentially predict when unrelated build failures occur. We conduct seven project-level experiments of Apache projects, where developers often comment on the validity of build failures, exerting efforts to verify whether these failures are unrelated to the current push.

To scale our study, we employ a keyword-detection approach to identify an initial set of *potential* 10,316 unrelated build failures. We then use the initial set of potentially unrelated build failures to manually curate a representative sample of 371 (based on 95% confidence level and 5% confidence interval out of 10,316 samples) developer-confirmed unrelated build failures. We perform a thematic analysis of this sample to understand why developers deem build failures as unrelated.

Once we understand the reasons why build failures are deemed as unrelated, we use this knowledge to extract features that can be used to predict unrelated build failures. Predicting unrelated build failures can help developers better understand their build notification. For example, the build notification can indicate that errors in a build are potentially unrelated to the developer's triggering push with an accompanying probability. Although it is important to maintain successful build statuses (Duvall et al. 2007), developers could make more informed decisions regarding how to proceed with the merging of contributions when an unrelated failure is present. Moreover, knowing that a build failure is unrelated to the current push may help the debugging process of the build failure as developers would know that the build failure is unrelated to the scope of the changes within the current push.

We use a semi-supervised approach known as *Positive and Unlabeled* (PU) learning (Liu et al. 2002; Li and Liu 2003; Yu et al. 2002; Nguyen et al. 2011; Yang et al. 2012; Li et al. 2010) to predict unrelated build failures. We develop PU-learning models (including the classic Elkan & Noto and weighted Elkan & Noto models) for each of our seven project-level experiments. Our semi-supervised models demonstrate robust performance, with median AUC values ranging from 0.62 to 0.97 across the seven projects analyzed, reflecting how

the datasets varied in complexity and patterns, creating different challenges for prediction. Moreover, the models perform better than random guessing and can provide a reasonable indication as to whether a build failure is related to the current code push. We then conduct a permutation feature importance analysis to identify the most influential features in our PU-learning models, following the methodologies established in previous studies (Rajbahadur et al. 2017; Tantithamthavorn et al. 2018; Jiarapakdee et al. 2020). Our findings and prediction model offer actionable guidance for developers when a build failure is predicted to be unrelated to recent code changes. Rather than being ignored, such failures should be triaged differently from code-related issues. Specifically, we recommend developers re-run the build in a controlled environment, inspect CI logs to investigate potential causes, and report persistent failures to the CI/infrastructure team. We elaborate on these recommendations in the Discussion section. This is consistent with recent recommendations by Santos et al. (2025), who emphasize the need to monitor CI practices to enhance both reliability and development efficiency. In the next section, we explore the challenge of *unrelated build failures*.

2 The Problem of Unrelated Build Failures

An unrelated build failure occurs when a build fails due to reasons that are unrelated to the current triggering push. Different from flaky tests (Bell et al. 2018; Luo et al. 2014; Eloussi 2015; Terragni et al. 2020; Palomba and Zaidman 2017; Silva et al. 2020; Thorve et al. 2018; Eck et al. 2019), unrelated failures are legitimate (as opposed to being intermittent) but are unrelated with the scope of changes of the current push. One could argue that unrelated build failures would not occur if CI practices are properly adopted, e.g., code should only be merged upon passing builds. Nevertheless, in projects where multiple developers work concurrently, as in our case and in the projects studied by Ghaleb et al. (2019a), one developer's change may inadvertently affect another's work. This can lead to unexpected build failures that are unrelated to the latter's contributions. Additionally, as highlighted by Felidré et al. (2019), projects using CI practices do not always adhere to the highest standards of CI practices, which can potentially cause an increased occurrence of unrelated build failures. As such, our research also acknowledges that achieving perfection in adhering to CI practices may not always be feasible in real-world scenarios.

For example, Fig. 1 depicts an unrelated build failure that occurred in the HIVE project (issue report HIVE-12446¹). On January 23, 2016, at 04:50, a build failure prompted an automatic comment posted by the QA bot. Approximately 19 hours later, developers confirmed the purpose of the push, which was to update the Tez version to 0.8.2 in both the `master` and `branch-2.0` branches. They then verified that the test failures were unrelated to the push and stated their intention to commit the modifications to the specified branches. Had developers known from the outset that the build failure was unrelated to their push, they could have saved time and effort by making their decision earlier in the development process.

We manually inspected a random sample of 383 build failures drawn from a total of 77,354 failure records across the seven studied projects, encompassing both related and unrelated failures, determined using a 95% confidence level and 5% confidence interval. Among the 383 randomly selected failures, we found 45 unrelated failures explicitly

¹<https://issues.apache.org/jira/browse/HIVE-12446>

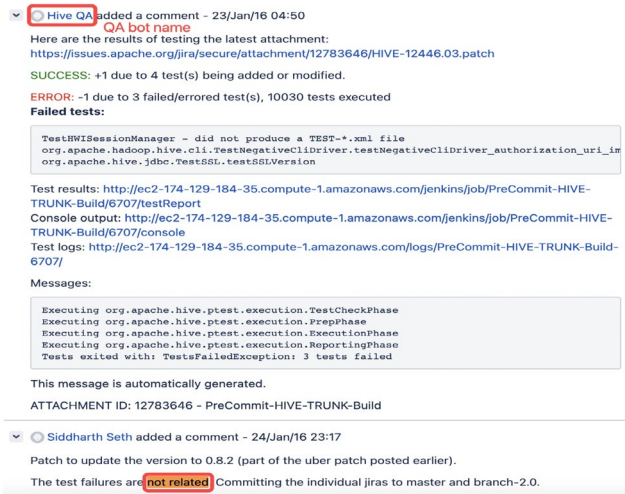


Fig. 1 An example of a CI bot comment in a historical issue report after the completion of the CI build

marked by developers using keywords such as “unrelated” and similar terms. This inspection confirmed that developers do indeed use terms like “unrelated”, “not relevant”, “not connected”, and similar language to describe unrelated failures in practice across our studied seven projects. This supports the validity of our keyword-based strategy for identifying such cases. However, we acknowledge that keyword usage may vary across projects and that not all unrelated failures are explicitly labeled. To address this, we distinguish between two types of unrelated failures: (1) those containing clear keywords that indicate the failure is not related to a code push, and (2) those without explicit indicators, leaving their cause ambiguous. The former can be identified through a Heuristic-based Labeling (HL) approach (explained below), whereas the latter are more challenging. Since failures not captured by keyword matches cannot be confidently regarded as unrelated, the setting is better suited to a semi-supervised learning paradigm. Therefore, we adopt a Positive–Unlabeled (PU) learning framework: failures with relevant keywords are treated as positive examples, while all others remain unlabeled, acknowledging that the absence of a keyword does not imply relevance to a code push. This design allows the model to benefit from weak supervision while remaining robust to inconsistent or incomplete developer annotations.

Heuristic-Based Labeling (HL) To increase the breadth of our understanding regarding unrelated build failures, we begin by adopting a naïve approach based on keywords to automatically identify build failures that are **potentially** unrelated. We call this approach *Heuristic-based Labeling*, which is shown in Fig. 2. Firstly, we create our list of keywords based on a sample of comments in which developers indicated that the build failure was not linked to their push. The set of keywords we use is: *not related*, *unrelated*, *irrelevant*, *not relevant*, *not connected*, *unconnected*, *not linked*, *uncorrelated*, *not correlated*. Next, we iterate over the historical comments in an issue report. Once there is a build failure comment (e.g. BC1 in Fig. 2), we analyze the following comments (e.g., C2 to C4 in Fig. 2) until the next build comment (e.g., BC2 in Fig. 2). If we identify the keywords listed above within the set of analyzed comments by developers, we flag the build failure (e.g. BC1 in Fig. 2) as **potentially** unrelated.

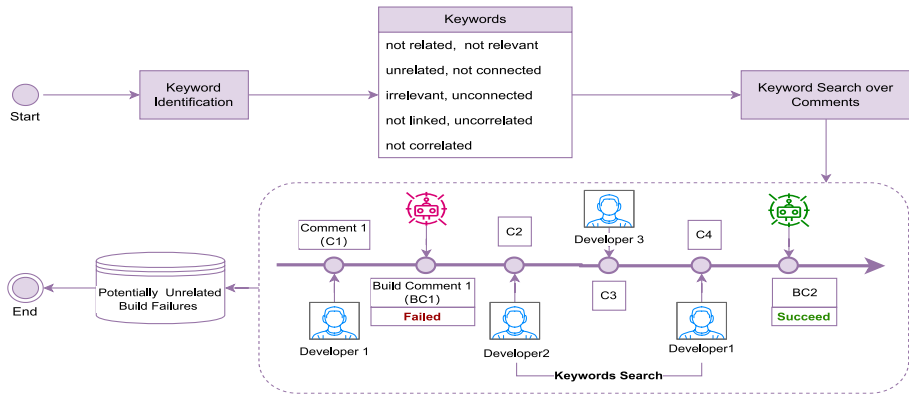


Fig. 2 Approach of Heuristic-based Labeling (HL)

Figure 3 shows an overview of our research process, which illustrates our end-to-end research process comprising five interconnected phases: 1) Project Selection (PS) identifies Apache projects with mature CI ecosystems. 2) Data Collection (DC) obtains build failures, issue reports, and developer comments. 3) Document Analysis (DA) examines 371 failures to derive why developers deem failures unrelated. 4) PU-Learning (PUL) trains semi-supervised models using features informed by (3). Feature Importance (FI) identifies key predictors of unrelated failures. We now detail these phases.

Project Selection (PS) The first step is concerned with the project selection process. For the purpose of studying CI projects, we have chosen ASF (Apache Software Foundation) projects as the CI culture has matured within the ASF since the early 2000s. In fact, the ASF is the birthplace of the first-ever Java CI tool—APACHE GUMP, which represented the nightly build of the entire Java stack.² ASF is also where ANT was developed, and its XML output ‘<junit>’ has become the standard format for JUNIT results. Furthermore, the HADOOP project introduced APACHE YETUS³, a tool designed to be launched whenever a pull request is made. YETUS runs tests, prints the results, and provides code veto.⁴ For instance, a push that has not been tested by YETUS would not proceed to code reviews. Considering the array of developments within the ASF regarding CI tooling and support for CI practices, we believe that ASF projects are suitable for our work.

Given that the ASF uses JIRA to manage its projects, we employed the JIRA API⁵ to extract the necessary metadata, such as issue reports and the main programming languages of all ASF open-source projects that are available (as shown in Table 1). ASF projects using a CI pipeline will normally employ a CI bot that generates automatic comments representing the build logs and statuses whenever a CI build is completed (see Fig. 1). We use these automatic comments to extract the CI build information of our ASF projects. For example, in the HIVE project, we collect all comments from the ‘Hive QA’ author, which represents the build runs of said proj-

²<https://gump.apache.org>

³<https://yetus.apache.org>

⁴<https://issues.apache.org/jira/browse/HADOOP-15281>

⁵<https://pypi.org/project/jira/>

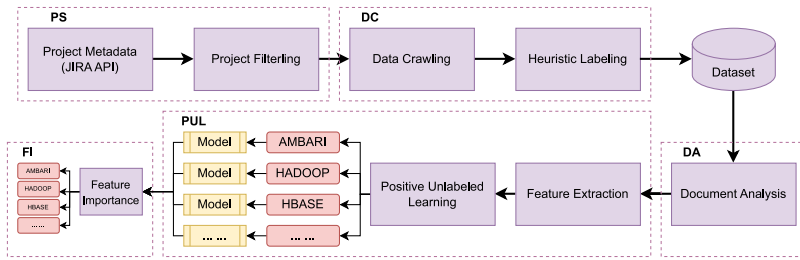


Fig. 3 The overview of the process in our study

ect. Given that different ASF projects assign different names for their QA bots, we collect the history of authors involved in the discussion comments of each project, searching for the ‘QA’ keyword. Typically the QA bot will follow the ‘{PROJECT NAME} QA’ pattern.

Table 1 shows the details of metadata we collect through JIRA API. The table shows the top 20 projects out of the 100 projects’ metadata we collected. The metadata for the remaining 80 projects can be found in our replication package for reference (Huang 2024). For each of the 100 projects, we check the total number of issue reports and whether the project contains CI build logs in their comments history. We arrange the projects in descending order in terms of the number of issue reports, since we seek to select projects with a larger amount of data as possible for further analyses. We restrict our focus to projects predominantly programmed in Java and exclude those projects lacking CI build logs on their issues. We also exclude projects that contain a limited number of CI build logs (i.e., less than 10%) since we wish to focus on the more active projects. As a result, from a total of 100 projects, we choose AMBARI, HBASE, HIVE, HADOOP, HDFS, YARN, HDDS for our studies. Although popular open-source projects such as Spark and Cassandra contain a large volume of issue reports, we excluded them due to the fragmented nature of their CI infrastructure and metadata. Unlike ASF projects, these projects often use cross platforms (e.g., GitHub Actions, CircleCI) and do not rely on a centralized issue tracker like JIRA. As a result, obtaining a consistent and complete dataset, including issue reports, CI outcomes, and

Table 1 Project metadata and CI/CD failures. The selected projects are highlighted in red

Project Name	Num of Issues	Languages	Hist CI Builds	Ratio of Issues Contain Build Comments	Ratio of Heuristic Unrelated build Failures
SPARK	24463	Scala			
AMBARI	21936	Java	YES	32.35% (>10%)	9.1%
ARROW	12514	C++			
CAMEL	11652	Java			
CASSANDRA	10672	Java			
BEAM	7927	Java			
IGNITE	7404	TypeScript			
HBASE	7327	Java	YES	44.45% (>10%)	10.62%
NIFI	6636	Java	YES	0.03%	
KAFKA	6505	Java			
HDFS	6421	JAVA	YES	53.22% (>10%)	17.52%
IMPALA	6379	C++	YES	0.04%	
MESOS	5310	C++			
KARAF	4972	Java			
HDDS	4780	Java	YES	20.91% (>10%)	11.11%
AMQ	4684	Go			
HADOOP	4562	Java	YES	46.62% (>10%)	17.70%
YARN	4449	Java	YES	63.15% (>10%)	16.73%
WICKET	4323	Java			
HIVE	4273	Java	YES	40.68% (>10%)	9.94%

source code patches, becomes significantly more difficult. To ensure consistent data collection and to reliably link CI build results with corresponding development activities, we limit our study to the seven ASF projects. Although seven projects represent a limited number compared to many empirical studies, especially in the area of mining software repositories, we treat each project as a separate case study. For example, this number of projects enables us to delve deeper and manually investigate why developers consider the build failures to be unrelated. Additionally, it allows us to build models for each individual project instead of creating a general model based on a large amount of data. Our objective is to showcase the viability of our approach in identifying unrelated build failures, rather than striving for an extensive level of generalizability in our findings at this stage.

By applying the heuristics labeling to our seven studied projects, we obtain a set of 10,316 potentially unrelated build failures. We then use this list of potentially unrelated build failures to curate a representative sample of 371 **developer-confirmed unrelated build failures** with a 95% confidence level and 5% confidence interval (more on the dataset curation in Section 3). By **developer-confirmed unrelated build failures** we mean build failures that developers explicitly stated as unrelated in their comments, and we manually verified that the comments indeed indicate the failure is unrelated to the current code push.

Table 2 summarizes the recognition delay and code complexity associated with unrelated build failures across our studied projects. To measure the recognition delay, we automatically scan all post-failure comments for a set of developers using the keywords (e.g., “not related”, “unrelated”, “irrelevant”, etc.) which we derived from an initial sample of true unrelated failures. When one of these keywords appears in a comment, we take that moment as the explicit confirmation that the failure was deemed unrelated. For each confirmed unrelated failure, we record the interval between the timestamp of the CI bot’s initial failure notification comment and the first developer comment containing one of our unrelated keywords set. We also measure the code churn of the push triggering the unrelated failure to understand the amount of code potentially withheld due to the unrelated failure. Figure 4 shows that, on the median, the time required from developers to determine that a failure is unrelated is around 4 hours.

Table 2 Summary of time misspent and code complexity for unrelated build failures. Time Misspent measures the interval between CI failure notification and developer confirmation of “unrelated”. Code Complexity metrics provide context for interpreting the time values

Project	N	Time Misspent (Hours)				Code Complexity [†]		
		Median	Mean	Total	Std	Churn	Files	CC
AMBARI	177	4.87	230.99	40,885	1,818	12	1	7
HADOOP	1,472	7.07	31.61	46,530	95	24	1	6
HBASE	1,503	5.24	32.60	48,998	102	114	2	8
HDDS	210	2.43	16.70	3,507	33	250	5	12
HDFS	2,936	6.99	118.11	346,771	701	104	2	6
HIVE	2,368	8.37	18.89	44,732	28	82	2	6
YARN	1,650	4.18	11.51	18,992	20	234	3	9
Total	10,316	5.24	53.36	550,415	709	104	2	7

N: Number of unrelated build failures identified by our heuristic approach.

[†]Code Complexity shows median values. Churn = lines of code changed in the code change push;

Files = modified files in the code change push;

CC = Cyclomatic Complexity calculated using `lizard` (<https://github.com/terryyin/lizard>) on associated code patches

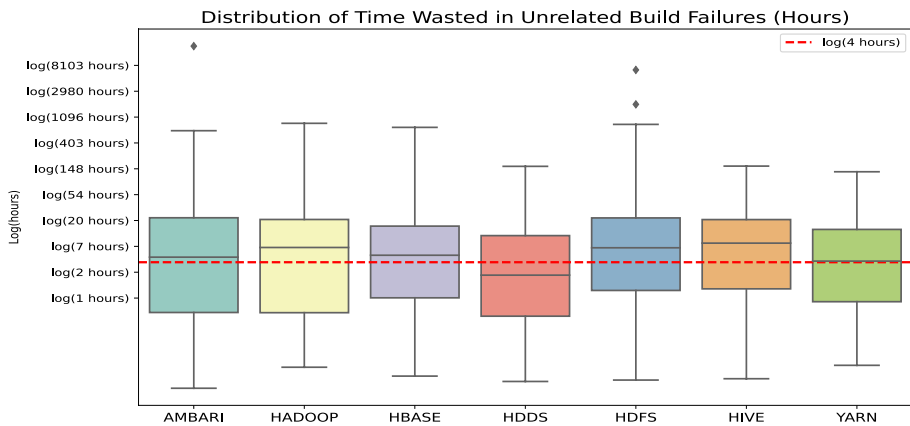


Fig. 4 Time misspent on identifying unrelated build failures

The code complexity analysis shows that unrelated failures have a median of 104 lines changed, 2 files modified, and a Cyclomatic Complexity of 7. The time interval between the CI failure appearance and developer confirmation spans several hours, with a median of 5.24 hours across all projects. While we cannot determine exactly how developers spent this time, whether actively investigating, waiting for additional builds, or attending to other tasks, the misspent time intervals suggest that confirming a failure as unrelated is not straightforward. This observation highlights the potential value of prediction model for relatedness of failures to the code changes. When developers receive a failure notification along with an indication that the failure is likely unrelated to their code changes, they can narrow down their investigation scope more quickly and make faster decisions about how to proceed.

This observation highlights the importance of automatically identifying whether a build failure is unrelated, helping developers with making the decision whether they should proceed with the merge or not. Additionally, knowing whether a build is unrelated beforehand may also help developers debug the breakage more quickly since the problem scope to be investigated would be reduced (i.e., the breakage is unrelated to the current push).

Developer Feedback Study To understand the practical significance of unrelated build failures, we conducted a developer feedback study. We distributed a short anonymous questionnaire to active contributors of the seven studied Apache projects, identified from JIRA commit histories and GitHub contributor lists. The questionnaire consisted of five questions covering (Q1) the frequency of encountering unrelated CI build failures, (Q2) the time spent investigating whether a failure is related, (Q3) the actions developers take when they determine a failure is unrelated, (Q4) the perceived usefulness of a prediction tool, and (Q5) an open-ended question for additional experiences and deeper explanations. A total of 297 invitation emails were sent.

We received five responses from experienced engineers contributing to projects in our study. Three respondents have 10+ years of CI/CD experience and two have 5–10 years. All five respondents encounter unrelated CI build failures at least a few times per month, with four out of five reporting that they encounter them multiple times per week. Three out of five respondents spend over 30 minutes per incident investigating whether a failure is related

to their changes, and the remaining two spend 10–30 minutes. The most common action is re-running the build (5/5), followed by ignoring the failure and proceeding with the merge (3/5) and investigating the root cause despite knowing it is unrelated (3/5). Four out of five respondents consider a prediction tool can be useful, with one rating it as very useful.

Three respondents provided open-ended responses that directly speak to why unrelated build failures matter to practitioners. One respondent (Hadoop, 10+ years) stated that “the unrelated build failures are a serious issue that should be prioritized. This directly confirms the practical significance of the problem from a developer’s perspective. Another respondent (Hadoop, 10+ years) described how unrelated failures persist even in projects with mature CI tooling, noting that the lack of CI coverage for certain components forces developers into burdensome manual pre-merge testing. These feedback validates that developers see value in automated build failure classification.

To automatically identify unrelated build failures, we begin by investigating the ratio of potentially unrelated build failures to build failures in our larger dataset (i.e., not the manually curated dataset). We observe that the ratio of potentially unrelated build failures to build failures is $\frac{10;316}{77;354} \approx 13.33\%$, meaning that the data is imbalanced. Additionally, it is also likely that there are several occasions where developers do not explicitly state when a build failure is unrelated. For instance, developers could have discussed the relatedness of a build failure on other communication channels (El Mezouar et al. 2021; Ehsan et al. 2020) as opposed to posting comments on the issue report. As such, it is likely that many positive cases (i.e., unrelated failures) cannot be retrieved from the available data. As such, both issues of i) imbalanced data and ii) unavailability of positive cases, motivate us to build a semi-supervised PU-learning approach, which is suitable for such challenges (Liu et al. 2002; Li and Liu 2003; Yu et al. 2002; Nguyen et al. 2011; Yang et al. 2012; Li et al. 2010). We detail our research methodology in the following section.

3 Research Methodology

Data Collection (DC) Following the project selection, we begin to collect further data from JIRA, including information from issue reports (e.g., issue priority, issue activities), pushes linked to each issue report, and comments associated with issue reports (including developer and CI bot comments). As briefly explained in Section 2, we develop a heuristic labeling approach to identify whether a CI build failure (as reported by the CI bot comments) is likely unrelated. First, for each issue report, we locate the CI bot comments based on the author of the comment (e.g., ‘Hadoop QA’). If the bot comment reports a build failure, our approach searches for relevant keywords in the following comments, such as “**unrelated**”, or “**irrelevant**”, that can indicate that the build failure is unrelated to the current push. We are aware that a heuristic-based approach such as this may likely yield sub-optimal accuracy. We accept this limitation at this stage, since our main goal is to strategically reduce our search space to then create a curated dataset, using the identified potentially unrelated failures as the initial input. Using the heuristic-based approach, we obtained 10,316 potentially unrelated build failures across our studied projects.

Document Analysis (DA) Once we obtain our set of potentially unrelated build failures in the DC step, we aim to further process the comments left by developers and understand the

reasons why build failures are deemed as unrelated. The first step of this analysis is to build a statistically representative sample of **developer-confirmed** unrelated failures (as opposed to **potentially** unrelated failures). For this purpose, we initially create a representative random sample of potentially unrelated failures across our seven projects. Considering a confidence level of 95% and a confidence interval of 5% on 10,316 unrelated build failures, we determine a sample size of 371. To populate our sample, we initially verify whether the build failures identified as potentially unrelated by the HL approach are indeed confirmed as unrelated by developers. We do this by manually confirming that the HL-identified comment following a build failure indicates that the failure is unrelated (i.e., a developer-confirmed unrelated build failure). Whenever we identify a false positive⁶, we replace it with another sample until we find one that is confirmed as unrelated by developers. To preserve the randomness of the process, whenever a false positive is detected, it is returned to the pool, and a new instance is randomly drawn from the entire pool of 10,316 instances. This verification and replacement procedure is repeated until we obtain 371 developer-confirmed unrelated failures. Two authors (i.e., A.H. and M.M.) worked together throughout the entirety of this step.

Once we assemble our sample, our goal is to further analyze the comments of developers to better understand why unrelated build failures are deemed as such. In general terms, comments from developers can be interpreted as digital documents describing a historical process behind an organization or project. As such, to elicit the knowledge embedded in these digital documents, we use a technique called document analysis (Bowen 2009; Zina et al. 2021), a qualitative approach to examine documents and yield data from them. This data, which can be quotations, excerpts, or entire passages, are then organized into major themes along with examples. We treat the CI bot comments (i.e., the build failure logs), their related information, and following discussions as the digital documents to be inquired in our document analysis. The **guiding question** for our document analysis is the following: “*Why do developers deem a build failure as unrelated to the current push?*”

Figure 5 shows an overview of the document analysis process. Three coders are involved in the document analysis process, all of whom have academic and industry backgrounds in software engineering.

First, the first author (i.e., “main coder” in Fig. 5) manually scrutinized each instance of our digital documents to answer the **guiding question** for each of these instances. The answer to the **guiding question** should be the reason why an unrelated build failure is deemed as such. For example, Eli Collins, a developer examining a build failure for issue HDFS-3582,⁷ states: “*Fixed one findbugs warning (FindBugs isn’t able to figure out one statement after terminate is currently unreachable). The other two are HDFS-3615. Test failures are unrelated.*” The fact that the failed tests are associated with other pushes made for different issues, leads us to identify the **external interference** theme as the main reason contributing to the unrelated nature of the build failure. This inductive approach yields a set of themes that express the common reasons why developers deem build failures as unrelated to the present push. These themes will be presented and analyzed in Section 4.

After generating and documenting the complete set of themes through multiple reflective iterations, the main coder engaged in discussions with two other authors (referred to as “secondary coders”) to refine the themes and their interpretations. The main coder gener-

⁶The comment from developers included keywords suggesting doubt about the build failure being related to the code push. However, this was later confirmed by others to indeed be related to the code push.

⁷<https://issues.apache.org/jira/browse/HDFS-3582>

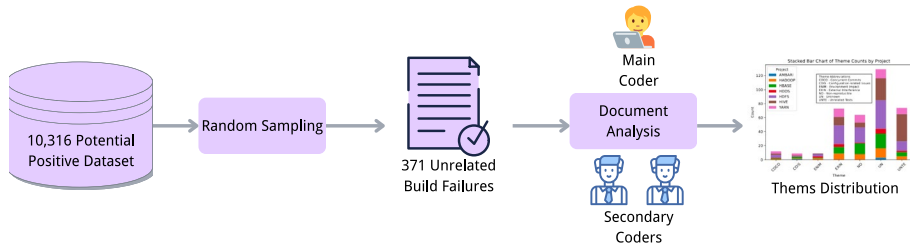


Fig. 5 An overview of the document analysis process

ated 7 themes with detailed explanations as shown in Section 4, i.e., **Results**. Then, The secondary coders performed a deductive analysis, using the seven themes established by the main coder. That is, the secondary coders independently labeled all the 371 instances using the set of themes established in the previous step. During this stage, the secondary coders had the opportunity to propose new themes. There are 2 additional themes created by one of the secondary coders, i.e., Time Out Issue, and Dependencies. However, after carefully discussion among coders, those two themes can be categorized as **environmental impact** and **external interference** respectively after we made the scope of themes more clear. We also use Cohen's Kappa to explore the level of agreement between the main coder and the secondary coders (Kitchenham and Brereton 2013; Dybå and Dingsøyr 2008). Cohen's Kappa is a statistical measure used to assess the agreement between the two raters on two raters on categorical data while correcting for chance agreement. It is widely used in software engineering studies that involve qualitative coding or manual classification (e.g., defect categorization, thematic analysis). We used the "cohen_kappa_score" method from the scikit-learn package to calculate Cohen's Kappa. We observed disagreements in 70 cases (18.92%), with a Cohen's Kappa score of 0.76 between the main coder (A.H.) and the secondary coder (M.M), indicating substantial agreement [4]. Similarly, A.H. and the secondary coder (D.A) disagreed in 67 cases (18.11%), achieving a Kappa score of 0.77, also reflecting substantial inter-rater reliability (Ajibode et al. 2025). After removing overlapping cases (where both secondary coders disagreed with the main coder on the same instance), 130 unique instances required consensus discussion, while 240 cases (64.9%) achieved full agreement among all three coders. The disagreements primarily arose from three sources: (1) *Conservative labeling* by the main coder when developer evidence was limited—in cases where developers confirmed a failure was "unrelated" but provided insufficient explanation, the main coder labeled the reason as **Unknown**, while secondary coders sometimes inferred more specific themes based on contextual clues in build logs or surrounding comments; (2) *Conceptual overlap* between adjacent themes, such as **Unrelated Tests** vs. **Non-reproducible** and **External Interference** vs. **Environment Impact**; and (3) *Multi-causal failure narratives* where coders prioritized different contributing factors mentioned in the same developer comment. For example, in issue HDFS-11274, a developer commented: "The test failure looks unrelated and didn't repro locally over ~ 10 attempts. It also failed in a few other pre-commit runs recently." The main coder labeled this as **Non-reproducible** based on the "didn't repro locally" statement, while one secondary coder labeled it as **Environment Impact** (timeout across multiple CI runs), and the other as **Unrelated Tests** (developer's explicit "unrelated" statement).

Through iterative consensus discussions, we refined our themes by merging similar proposals (e.g., multiple timeout-related ideas into a unified **environmental impact** category and consolidating overly narrow labels (e.g., subsuming “Jenkins failures” under Configuration Issues). Finally, all coders engaged in online plenary sessions totaling around 65 hours, where they discussed the generated themes and the labeling of instances. By the end of these plenary sessions, the new themes were merged, and labeling conflicts were resolved, resulting in the final set of themes. This final set of themes is described in Section 4.

Feature Engineering (FE) After studying the reasons why developers deem certain build failures as unrelated in the **DA step**, we use this knowledge to engineer features for our intended machine learning approach (Yue et al. 2024b, a). From build failures, we extract 33 features based on issue reports, preceding comments, and the triggering push.

Table 3 summarizes the 33 features organized into three dimensions: 19 features from issue reports, 3 features from historical comments, and 11 features from the triggering push. Detailed calculation formulas and extended rationales for each feature are provided in Appendix A.

We also remove collinear features as they interfere with each other’s significance within the model (Huang et al. 2019; McIntosh and Kamei 2018; McIntosh et al. 2016). We use Spearman’s ρ rank correlation on our 33 features for each project. Similar to previous work (Huang et al. 2019; Ghaleb et al. 2019a; Gousios et al. 2014), we remove features if $|\rho| \geq 0.7$. In other words, if a pair of features exhibits a correlation $|\rho|$ greater than 0.7, we prefer the simple and more informative metric over the complex metric (Vandekerckhove et al. 2015; Ghaleb et al. 2022). Figure 6 shows an example of Spearman’s rank analysis on features in AMBARI. Based on the analysis, we retain the feature *Number of Similar Failures* and remove *Is Shared Same Emsg*, as the former provides a more informative signal. Similarly, we retain *Modified Source Code Files* and discard highly correlated features including *Source Code Lines Modified*, *Source Code Lines Added*, *Has Source Code*, and *Source Code Lines Deleted*. In the case of configuration-related features, we preserve *Modified Config Files* while removing *Config Lines Modified*, *Has Config Files*, and *Config Lines Added*, in accordance with the correlation results. Further details on feature selection and Spearman’s rank correlation analysis for the remaining studied projects can be found in our replication package (Huang 2024).

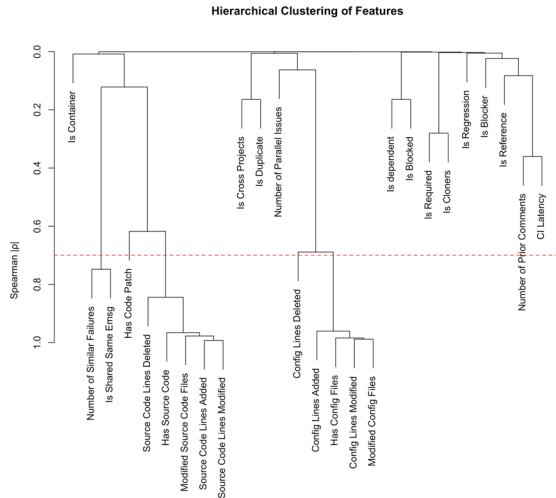
Also, if a feature can be predicted by other variables, it may lead to multicollinearity (Hanley and McNeil 1982). To mitigate multicollinearity, we conduct a redundancy analysis on the remaining features. We use the *redun* function in *R* to remove such redundant features if $R^2 > 0.9$. Our *redun* analysis does not identify any redundant variables. We perform this filtering on a per-project basis because multicollinearity depends on the empirical distribution of each dataset. In practice, feature distributions vary across projects. For instance, some projects make heavy use of certain issue-link types while others rarely use them, which affects how correlated those features are with other variables. Since our models are trained within each project, filtering within each project keeps the methodology internally consistent. This project-wise approach is also commonly adopted in prior empirical software engineering studies that build prediction models at the project level (Tantithamthavorn et al. 2018; Jiarpakdee et al. 2020; McIntosh et al. 2016).

Table 3 Summary of extracted features across three dimensions. Detailed calculation formulas are provided in Appendix A.

Feature	Description	Rationale
Issue Report Level (19 features)		
Priority	Urgency or importance of an issue report in Jira	Urgent issues attract rushed changes
Number of Parallel Issues	Count of issues worked on simultaneously (Perry et al. 2001)	Concurrent work may cause interference
Is Cross Projects	Whether issue relates to multiple projects	Cross-project changes increase complexity (Jayasuriya et al. 2025)
Is Duplicate, Is Blocker, Is Blocked, Is Regression, Is Dependent, Is Incorporates, Is Required, Is Reference, Is Completes, Is Testing, Is Issue Split, Is Supercedes, Is Cloner, Is Container, Is Parent Feature, Is Child-Issue (16 Jira link-type features)	Boolean flags indicating various issue relationships and dependencies in Jira's "Issue Links" section	Issue interdependencies can lead to incomplete implementations or conflicts causing unrelated failures (Lüders et al. 2022a, b)
Historical Comment Level (3 features)		
Number of Prior Comments	Count of comments before the build failure	Extensive discussion aids failure analysis
Is Shared Same Emsg	Whether error message matches previous failures	Shared errors suggest inherited issues
Number of Similar Failures	Count of failures sharing failed test/exception classes	Similar failures may share root causes
Push Level (11 features)		
CI Latency	Duration between push and build trigger	Long latency obscures cause-effect links
Has Code Patch	Whether build failure includes code patch	Missing patches hinder root cause analysis
Has Config Files	Whether patch modifies config files (.yaml, .xml, .properties) (Ghaleb et al. 2019a)	Config changes can cause misconfigurations
Config Lines Deleted/Added/Modified	Line counts for config file changes (Ghaleb et al. 2019a)	Config modifications affect dependencies
Has Source Code	Whether patch modifies Java source files (Ghaleb et al. 2019a)	Absence suggests config-related failures
Source Code Lines Added/Deleted/Modified	Line counts for Java file changes (Ghaleb et al. 2019a; Pan and Pradel 2021; Saidani et al. 2020a)	Extensive changes obscure failure causes
Modified Source Code Files	Count of modified Java files	Fewer files ease failure attribution

Positive Unlabeled Learning (PUL) Although a supervised binary classification approach might seem suitable, our dataset poses a key challenge: Positive instances (i.e., unrelated build failures) cannot be reliably identified with sufficient completeness or accuracy. In terms of completeness, developers do not always confirm whether a build failure is unrelated to code changes, leaving many true positives hidden among the non-positive samples.

Fig. 6 Dendrogram of hierarchical clustering based on Spearman correlation coefficients among features in the AMBARI project. The tree structure groups features by similarity, and the red dashed line marks the correlation threshold of $|\rho| = 0.7$. Features clustered above this threshold (below the red dashed line) were carefully reviewed, and less informative or redundant ones were removed to mitigate multicollinearity



In terms of accuracy, our heuristically labeled positives may include false positives, meaning that not all labeled unrelated failures are guaranteed to be correct. To illustrate, consider the issue of under-reporting smoking among pregnant populations (Graham and Owen 2003), where a significant number of smokers feel pressured to identify as non-smokers. In such cases, trusting the negative instances (i.e., non-smokers) is unreliable due to the presence of positive instances within them. The problem of unrelated build failures shares a similarity in that developers may not consistently and explicitly declare that a build failure is (un)related. Since the reliability of negative instances cannot be guaranteed, because the absence of evidence that a build failure is unrelated does not imply that the failure is truly related, yet we do have access to a trustworthy subset of positive instances, this setting is well suited for developing an approach based on Positive-Unlabeled (PU) learning (Elkan and Noto 2008; Fusilier et al. 2015; Hsieh et al. 2015). We treat each of our studied projects as a case study and build a model for each of them.

In our scenario where the accuracy of deciding a build failure is related to the current code push (negative instances) is unreliable, however, a subset of unrelated build failures (positive instances) is available with high confidence, as 371 positive samples were manually investigated by coders in DA process. Positive-unlabeled (PU) learning provides an effective framework for addressing the challenge. It is an approach that addresses scenarios where only a small subset of positive instances is labeled, and the rest of the data is unlabeled, potentially containing both positive and negative examples. To build our semi-supervised learning models, we adopt two distinct variants of the PU classifier, i) the Classic Elkan & Noto classifier and ii) the Weighted Elkan & Noto classifier (Elkan and Noto 2008). The classic and weighted Elkan & Noto classifiers operate under assumptions of *SCAR*. The Selected Completely at Random (*SCAR*) assumption (Teisseyre et al. 2024) posits that among all the positive examples in our dataset, each one has an equal and unbiased chance of being labeled, regardless of its specific features or attributes. In simpler terms, the labeling of positive examples happens entirely by chance and is not influenced by any characteristics of the data. Applying this assumption allows us to treat the randomly chosen and labeled positive examples in DA as a representative sample of all positive instances. It enables us to build a predictive model to learn from this unbiased sample and estimates

the true likelihood of an example being positive, even when faced with a large number of unlabeled or ambiguous cases. Formally, the SCAR can be expressed as

$$p(s = 1 \mid x, y = 1) = p(s = 1 \mid y = 1) = c \quad (1)$$

where:

- $s = 1$ denotes a sample being labeled,
- $y = 1$ denotes a sample being truly positive,
- $c = p(s = 1 \mid y = 1)$ is a constant probability, representing the chance that any positive example is labeled.

The classic Elkan & Noto classifier assumes that unlabeled data can be treated as a general mixture of positive and negative instances. To build the classic Elkan & Noto model, it first trains a nontraditional classifier $g(x)$ using the labeled positive examples and the unlabeled data. This classifier estimates the probability that a sample is labeled given its features, $p(s = 1 \mid x)$. The target is to learn a function $f(x)$, where $f(x) = p(y = 1 \mid x)$. Under the SCAR assumption, the relationship between this probability of a sample is labeled $p(s = 1 \mid x)$ and the true probability of an example being positive, $p(y = 1 \mid x)$, is given by:

$$f(x) = p(y = 1 \mid x) = \frac{p(s = 1 \mid x)}{c} = \frac{g(x)}{c} \quad (2)$$

By applying this method, the Classic Elkan & Noto classifier effectively learns from the available data to predict the true probability $p(y = 1 \mid x)$, even in the absence of negative labels. The Weighted Elkan & Noto classifier extends the Classic Elkan & Noto classifier to account for situations where the unlabeled data may not perfectly represent the negative class. The key insight of this method is that the unlabeled data cannot be treated as entirely negative. Instead, each unlabeled example is viewed as both a potential positive and negative instance, with weights assigned to each interpretation. The weights are derived using the probability of an example being positive given it is unlabeled, denoted as $p(y = 1 \mid x, s = 0)$. These weights are computed based on the relationship between the probabilities of labeling and the underlying class distribution, expressed as:

$$w(x) = \frac{(1 - c)}{c} \cdot \frac{p(s = 1 \mid x)}{1 - p(s = 1 \mid x)} \quad (3)$$

Here, $c = p(s = 1 \mid y = 1)$ represents the probability of labeling for a positive instance, a constant under the SCAR assumption that positive examples are selected completely at random. During the training process, each sample is assigned with two roles, i) positive example with weight $w(x)$ and ii) a negative example with complementary weight $1 - w(x)$. These two PU Learning methods play a crucial role in scenarios where collecting negative examples is difficult or infeasible. We can leverage only positive and unlabeled data, and develop effective predictive models for positive data (which are build failures unrelated to code pushes in this paper).

Evaluating models based on PU learning poses a challenge due to the absence of truly negative instances. We only know the true class label of a sample of positive instances (i.e., 371 developer-confirmed unrelated build failures that are used in **DA**). The remaining build failures are not determined whether is related to code push. Normally, the presence of unlabeled instances would not be possible to compute metrics such as true positive rate, false negative rate, precision, recall, and F-measure (Japkowicz and Shah 2011). To address this, existing studies (Japkowicz and Shah 2011; Elkan and Noto 2008) commonly adopt an approach where a subset of positive instances is intentionally hidden within the negative set, thereby constructing an unlabeled set in which true negatives and hidden positives are treated as ‘unlabeled’. Hence, the model is trained on PU data but evaluated on fully labeled data, and evaluation metrics, like precision and recall, can be accurately calculated.

Progress for Constructing Evaluation Datasets P, Q, N Figure 7 provides a visual representation of the dataset structure employed in this study, a standard procedure inspired by prior research in PU learning (Elkan and Noto 2008). This methodology defines three distinct subsets of data: *P*, *Q*, and *N*. *P* is a subset consisting of explicitly labeled positive instances, representing data points that are definitively identified as positive based on prior knowledge (Elkan and Noto 2008). In the Heuristic-based labeling step, we obtain 10,316 positive samples, and we already have 371 samples from the **DA** step, which have already been confirmed as unrelated build failures (positive samples). During the DA process, we established explicit, predefined criteria through dual-coder consensus. These criteria were consistently applied throughout the extended labeling process to ensure continuity and objectivity. The main coder in the DA step further manually labels each project to extend the positive sample size to 100 per project. To ensure reliability, these 100 positive samples were independently validated by a second coder, and the two coders achieved a near-perfect Cohen’s Kappa of 0.98. The high agreement is due to the simplicity of the task, which involves checking whether developers explicitly stated that a failure is unrelated, leaving little room for inter-

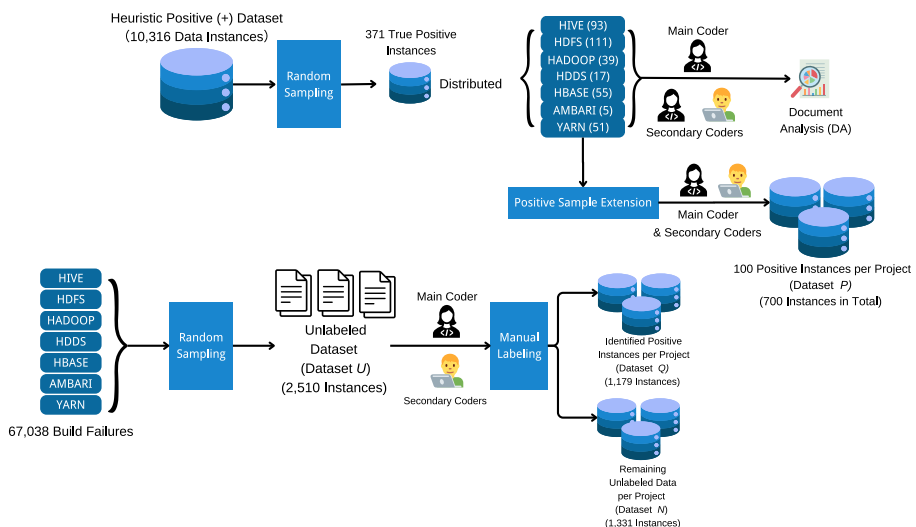


Fig. 7 The Process Flow for Constructing the P, Q, and N Datasets

pretation. As a result, we curated $100 * 7 = 700$ positive samples in total as the dataset P . Those 700 positive samples overlap with the 371 samples used in the DA step.

Dataset U consists of subsets Q and N . We randomly select samples from each project's full dataset for manual labeling. The sample size for each project was determined using a confidence level of 95% with a confidence interval of 5%, ensuring that the selected samples are representative of the overall dataset U for each project. During the manual labeling process, two coders independently reviewed the failure logs and follow-up developer comments to determine whether each failure was confirmed as unrelated by developers, achieving a near-perfect agreement with a Cohen's Kappa of 0.91. The labeled positive samples from U are curated as dataset Q , and the rest of the data are called dataset N . Table 4 shows the detailed number of randomly selected samples for each project. As a result, we obtained 1,179 samples in Q and 1,331 in N , respectively.

SCAR Assumption Validation Previous work (Teisseyre et al. 2024) proposed a statistical test to verify whether a dataset satisfies the SCAR (Selected Completely At Random) assumption, which is essential for valid PU-learning. We adopt their method and apply it to the datasets of seven projects. In our setting, we treat the 700 positively labeled samples (identified by heuristics and manually confirmed) as known positives P , while the remaining samples are considered unlabeled U . Following (Teisseyre et al. 2024), we first approximate the class prior π (the proportion of positives in the entire dataset) using the ratio of manually confirmed positives Q to the unlabeled dataset U . A simple probabilistic classifier (e.g., RandomForest) is then trained with P as positives and a random subset of U (assumed mostly negative) as negatives, and applied to the rest of U to estimate the probability of being positive for each instance. Based on these scores and π , we derive an estimated positive set. To mimic the SCAR mechanism, we randomly relabel instances within this estimated positive set with probability $c = P(S = 1)$, where $P(S = 1)$ is the empirical proportion of labeled positives among all estimated positives. This simulates a random selection of labeled positives under the SCAR assumption. We then compute a test statistic that measures the discrepancy between the known positives (P) and the estimated

Table 4 Distribution of U , P , Q , and N among the seven studied projects

Project	Heuristic labeling			Manual labeling					
	Total Samples	Heuristic Positive Sample	Unlabeled Ratio [†]	Sample Distribution in DA Step	U^*	P	Q	N	Unlabeled Ratio [‡]
AMBARI	1950	177	0.909	5	322	100	246	76	0.236
HADOOP	8649	1472	0.830	39	368	100	130	238	0.647
HBASE	14160	1503	0.894	55	375	100	191	184	0.491
HDDS	1878	210	0.888	17	320	100	149	171	0.534
HDFS	16896	2936	0.826	111	376	100	166	210	0.559
HIVE	23885	2368	0.901	93	379	100	127	252	0.665
YARN	9936	1650	0.834	51	370	100	170	200	0.541
Total	77354	10316	0.867	371	2510	700	1179	1331	0.530

* Dataset U is randomly selected from the total samples in each project at a 95% confidence level with a 5% confidence interval.

[†] The unlabeled ratio next to the heuristic positive samples is computed as (Total Samples - Heuristic Positive Sample) / Total Samples)

[‡] The unlabeled ratio in the rightmost column is computed as $(U - Q) / U$

positives. Specifically, another classifier is trained to distinguish between the two groups, and the AUC is used as the statistic. A value close to 0.5 indicates similarity. To obtain a null distribution under the SCAR hypothesis, we repeat this procedure $B=1000$ times, and compare the observed statistic against this distribution to calculate a p -value. If the p -value exceeds a significance threshold (e.g., 0.05), we fail to reject the null hypothesis and regard the SCAR assumption as plausible. Otherwise, we reject it. Across all seven projects, the resulting p -values are consistently greater than 0.05. This indicates that we do not find statistical evidence against the SCAR assumption in our datasets, suggesting that SCAR is a plausible working assumption and providing justification for applying PU learning in our study.

To evaluate the effectiveness of our PU-learning models, we compare them against three simple yet informative baselines: i) Random Model: This baseline randomly assigns labels (unrelated or related) to instances, serving as a reference point that reflects performance expected by chance. ii) This baseline naively predicts all instances as unrelated (i.e., positive class). While simplistic, it serves as a useful comparison to assess how much benefit our model provides over always assuming unrelated failures. iii) Heuristic from Prior Error Messages (HPEM) approach, it aims to identify unrelated build failures by checking whether the error message matches that of the most recent failed build.

By following the evaluation approach inspired by evaluation settings in the prior work (Elkan and Noto 2008), our experiments involve the comparison of our PU-learning models against 3 baselines: (i) Random Model on P versus $Q \cup N$, (ii) Consistently Positive Model on P versus $Q \cup N$, (iii) HPEM on P versus $Q \cup N$, (iv) Classic Elkan & Noto Classifier on P versus $Q \cup N$, (v) Weighted Elkan & Noto Classifier on P versus $Q \cup N$. For each of the four methods, we perform 10-fold cross validation, and obtain a combined confusion matrix from the ten testing subsets. With one confusion matrix for each approach, we compute its recall and precision, f1 score and AUC as evaluation metrics. The rationale for using the confusion matrix is based on prior work (Elkan and Noto 2008), which employed a combined confusion matrix in their PU learning evaluation. Since the test set in each fold is relatively small, the resulting confusion matrices may be affected by sampling variability. To mitigate this issue, a combined confusion matrix is more suitable for evaluation. Particularly in scenarios where only positive labels are available, the aggregation of metrics provides a more stable and reliable basis for calculating evaluation and performance metrics (Forman and Scholz 2010). Cross-validation proceeds as follows. Partition P , Q , and N randomly into ten subsets each of size as equal as possible. For each of the ten trials, reserve one subset of P , Q , and N for testing. In each trial number i , this classifier is applied to the testing subsets P_i , Q_i , and N_i , yielding a confusion matrix of the following form:

	positive	negative	predicted
$P_i \cup Q_i$	a_i	b_i	
N_i	c_i	d_i	
actual			

The combined confusion matrix reported for each approach has entries $a = \sum_{i=1}^{10} a_i$ etc. In this matrix a is the number of true positives, b is the number of false negatives, c is the number of false positives, and d is the number of true negatives. Finally, precision is defined as $p = \frac{a}{a+c}$ and recall as $r = \frac{a}{a+b}$. The F-score is calculated as $F_1 = 2 \cdot \frac{p \cdot r}{p+r}$. Each of the four models yields a classifier that assigns numerical scores to the test data points, so

for each model, we can measure its AUC values by true positive rate (TPR, or sensitivity) against false positive rate (FPR, or 1 - specificity (or TNR)).

To further verify the robustness of our evaluation approach, we conducted an additional analysis using repeated stratified random sampling. Specifically, we repeated the 10-fold cross-validation process 30 times, where each repeat uses a different random seed to shuffle the data and produce a distinct fold assignment. Within each repeat, instead of combining all fold predictions into one confusion matrix, we computed Precision, Recall, F1, and AUC separately for each of the 10 folds, then averaged across folds. This per-fold averaging approach directly addresses the concern that aggregating into one confusion matrix may mask fold-level variability. The detailed results of this robustness analysis are presented in Section 4.3.

Feature Importance (FI) After evaluating the performance of our models in the **PUL** step, we study which features play an important role in detecting whether a build failure is unrelated. To compute feature importance, we apply the *permutation feature importance* method which is particularly beneficial when dealing with non-linear or complex estimators (i.e., tree model) that lack transparency (Breiman 2001). Our based estimator in the classic and weighted PU classifier is Random Forest, which allows us to calculate each feature's permutation feature importance value based on the trained PU classifiers. The fundamental idea behind permutation feature importance is to assess the contribution of a single feature by measuring how much the model's performance decreases when the values of that feature are randomly shuffled. In our case, we use recall and F1-score as the performance metrics. This technique has been widely used in previous studies (Rajbahadur et al. 2017; Tantithamthavorn et al. 2018; Jiarapakdee et al. 2020; Breiman 2001).

To compute the feature importance for each project, we collect feature importance values for each feature during each round of a 10-fold cross-validation process. For every fold, the corresponding feature importance scores are recorded. Once the cross-validation is finished, we have a set of feature importance values for each feature. We then calculate the median value of the feature importance values across these 10 rounds. By using the median, our ranking is less influenced by outliers and better reflects the central tendency of feature importance across all folds. Finally, we rank the features based on their median feature importance values, ordering them from the highest to the lowest to identify the most influential features in our model.

The computation of permutation importance allows us to identify which features contribute most significantly to the generalization capability of the examined model, not to mention that features that demonstrate importance on the training set but fail to do so on the held-out set could potentially lead to model overfitting.

By examining feature importance, we gain insights into how each feature contributes to the model's decision-making process. Understanding the relative significance of different features helps us better understand the relationship between these features and unrelated build failures, providing potential insights for practitioners and researchers.

Cross-Project Validation (CPV) A practice of training a model on data from one or more software projects and then validating or testing it on entirely different projects (Herbold et al. 2018). The goal of CPV is to assess how well our model can generalize across projects, which is especially helpful for transferring our PU Learning model to a new project

that lacks sufficient historical data. In our study, we adopt a leave-one-project-out design. For each of the seven studied projects, we designate it as the testing project and use the data from the other remaining six projects for training. This protocol is widely used in prior research (Zimmermann et al. 2009; Turhan et al. 2009), as it provides a strict generalizability test compared to within-project validation, ensuring that project-specific metrics do not inflate the performance of models.

4 Results

4.1 Developer reasons for deeming CI failures as unrelated.

Table 5 shows the themes generated in our thematic analysis during the DA step along with their descriptions. Figure 8 shows the distribution of themes across the studied projects. Unfortunately, a significant portion (34.86% , $\frac{129}{371}$) of unrelated failures are deemed as such for reasons **unspecified by the developer** to us, i.e., developers simply state that “*the failure is unrelated*” without providing further context or details.

On the other hand, the **unrelated tests** (20% , $\frac{74}{371}$) theme is the second most-frequent reason for developers deeming a failure as unrelated. This is the case when developers check that the failing tests are unrelated to the scope of the current push. For instance, in issue YARN-1410,⁸ a developer declares that the “*test case failure is not related.*” Such cases may occur due to several reasons, the presence of previous and unfixed broken builds, which would go against CI best practices but still occur in practice (Duvall et al. 2007; Felidré et al. 2019).

The third and fourth most frequent reasons for deeming build failures as unrelated are represented by the **external interference** (19% , $\frac{73}{371}$) and **non-reproducible** (17% , $\frac{64}{371}$) themes, respectively. The **external interference** theme describes build failures influenced by factors beyond the current code changes. These may include code pushes intended to address issues from other tasks or external modifications to dependencies on external resources. The **non-reproducible** theme represents situations when developers, to the best of their efforts, cannot reproduce the failure that is reported by the CI bot. These failures may occur due to external factors, such as differences in environments, system load, and more.

As Fig. 8 shows, the other themes are not as frequent as the top four themes mentioned above. However, they also represent interesting scenarios, such as the **configuration-related issues** theme, which represents situations where the failure is associated to faulty configuration settings that are unrelated to the current push.

Overall, our themes highlight that the challenges associated with unrelated build failures are tightly related to the concurrent nature of software development where several changes to the software are applied concurrently (Table 6).

To further understand the impact of unrelated build failures on developers, we categorized the 370 samples based on whether the failure could be resolved by the current developer modifying their pushed code. This categorization was performed by the first author and validated by the second author through two plenary sessions (approximately 3 hours total), with disagreements resolved through discussion until consensus was reached.

⁸ <https://issues.apache.org/jira/browse/YARN-1410>

Table 5 The explanation of inductive themes from document analysis

Theme	Description	Ratio (Amount)
<i>Concurrent Commits</i>	Refers to the occurrence of parallel commits in a CI environment. It can generate unpredictable errors in the codebase due to the unforeseen interactions between the changes.	3.24% (12)
<i>Configuration-related Issues</i>	Refers to failures or problems encountered by a QA bot or any other system due to incorrect or faulty configuration settings. These issues are unrelated to the code pushes or topics addressed in the current issue report.	2.43% (9)
<i>External Interference</i>	Failures that occur when dependencies or external resources, such as third-party libraries or package registries, are updated, unavailable, or behave unexpectedly. For example, a build fails because a new version of a JSON-parsing library introduces a breaking change, even though the current code patch did not modify anything related to that library.	19.73% (73)
<i>Environment Impact</i>	Refers to failures or issues that occur in a software development or testing process due to the environmental factors, such as Jenkins, Jira, timeout during the CI process, or already used addresses, and so forth	2.43% (9)
<i>Non-reproducible</i>	Failures that pass when re-running the same commit on a fresh environment, often due to transient infrastructure problems (e.g., disk corruption, network timeouts, clock skew, container cold starts). For instance, a build passes locally and on a re-run in CI, but would fail on another environment due to different configurations that are not easily apparent initially.	17.03% (64)
<i>Unrelated Tests</i>	Refers to errors or failures that exist in testing classes, which are unrelated to the changes introduced in the current push.	20.00% (74)
<i>Unspecified by the Developer</i>	Refers to any build failures that are reported to be unrelated to the current push but were not discussed in detail, so the reason for deeming a build failure as unrelated could not be derived.	34.86% (129)

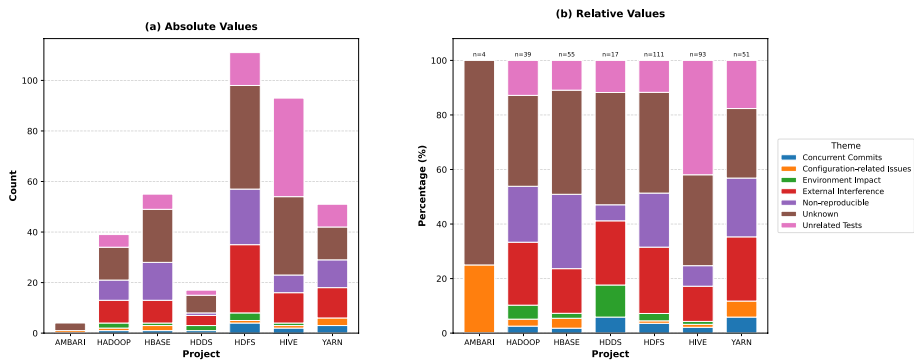


Fig. 8 The distribution of representative samples (371) across each theme and project. (a) Absolute values showing the count of samples per theme in each project. (b) Relative values showing the percentage distribution of themes within each project, with sample sizes (n) annotated above each bar

Our analysis reveals that 65.1% ($n=241$) of unrelated build failures represent cases where developers are blocked by issues they either cannot fix through code modifications (39.5%) or did not cause (25.7%). The first category (39.5%, $n=146$) includes **non-reproducible** failures caused by transient infrastructure problems ($n=64$), **external interference** from third-party dependencies ($n=73$), and environment impact requiring DevOps intervention ($n=9$). The second category (25.7%, $n=95$) includes **concurrent commits** from other developers ($n=12$), **unrelated tests** that are pre-existing failures ($n=74$), and **configuration-related issues** unrelated to current changes ($n=9$). The remaining 34.9% ($n=129$) are cases where developers explicitly stated the failure was unrelated and proceeded to merge their code despite the build failure, though they did not provide detailed explanations for the root cause. This merge-despite-failure behavior itself demonstrates that developers recognized these failures as unrelated to their changes and chose to proceed rather than investigate further.

This result provides empirical evidence that unrelated build failures do “wrongly block” developers, like making developers to spend time investigating build failures which either cannot be resolved by associated code changes or the push code changes did not cause the failure in the majority of cases.

4.2 PU-Learning Approach to Detect Unrelated Build Failures

Figure 9 shows the performance boxplots of our models across each project, evaluated using four key metrics: Precision, Recall, F1, and AUC. Each subplot represents the distribution of scores for our selected four different models, which showcases the variations in model effectiveness. Table 7 displays the details of the performance in each model (by median and $\text{mean} \pm \text{std}$ values) for our studied projects. We observe that our classic and weighted Elkan & Noto model consistently outperforms baseline models (Random Model, Constant Positive Model and HPEM approach) across most metrics (i.e., Precision, F1, AUC.). Specifically, the AUC values for the weighted Elkan & Noto model range from 0.62 (at HIVE) to 0.97 (at AMBARI), showcasing its ability to predict the unrelated build failures among a set of unlabeled data (Çorbacıoğlu and Aksel 2023). In the AMBARI project, the AUC value for the weighted Elkan & Noto model can reach as high as 0.97. One possible reason is that

Table 6 Theme distribution by project

Project	Theme	Count	%
AMBARI	Unspecified by the Developer	3	75.00
	Configuration-related Issues	1	25.00
HADOOP	Unspecified by the Developer	13	33.33
	External Interference	9	23.08
	Non-reproducible	8	20.51
	Unrelated Tests	5	12.82
	Environment Impact	2	5.13
	Concurrent Commits	1	2.56
	Configuration-related Issues	1	2.56
HBASE	Unspecified by the Developer	21	38.18
	Non-reproducible	15	27.27
	External Interference	9	16.36
	Unrelated Tests	6	10.91
	Configuration-related Issues	2	3.64
	Concurrent Commits	1	1.82
	Environment Impact	1	1.82
HDDS	Unspecified by the Developer	7	41.18
	External Interference	4	23.53
	Environment Impact	2	11.76
	Unrelated Tests	2	11.76
	Concurrent Commits	1	5.88
	Non-reproducible	1	5.88
HDFS	Unspecified by the Developer	41	36.94
	External Interference	27	24.32
	Non-reproducible	22	19.82
	Unrelated Tests	13	11.71
	Concurrent Commits	4	3.60
	Environment Impact	3	2.70
	Configuration-related Issues	1	0.90
HIVE	Unrelated Tests	39	41.94
	Unspecified by the Developer	31	33.33
	External Interference	12	12.90
	Non-reproducible	7	7.53
	Concurrent Commits	2	2.15
	Configuration-related Issues	1	1.08
	Environment Impact	1	1.08
YARN	Unspecified by the Developer	13	25.49
	External Interference	12	23.53
	Non-reproducible	11	21.57
	Unrelated Tests	9	17.65
	Concurrent Commits	3	5.88
	Configuration-related Issues	3	5.88

AMBARI has a highly imbalanced class distribution, with positive instances comprising 82.55% of the data. Hence, the AUC value can benefit from the weighted adaptation of PU learning, optimizing the model's focus on more significant positive samples. In projects like HDFS and YARN, the AUC scores of the weighted Elkan & Noto model maintain high median values (The value of AUC is 0.85 at HDFS and 0.81 at YARN) but are slightly lower

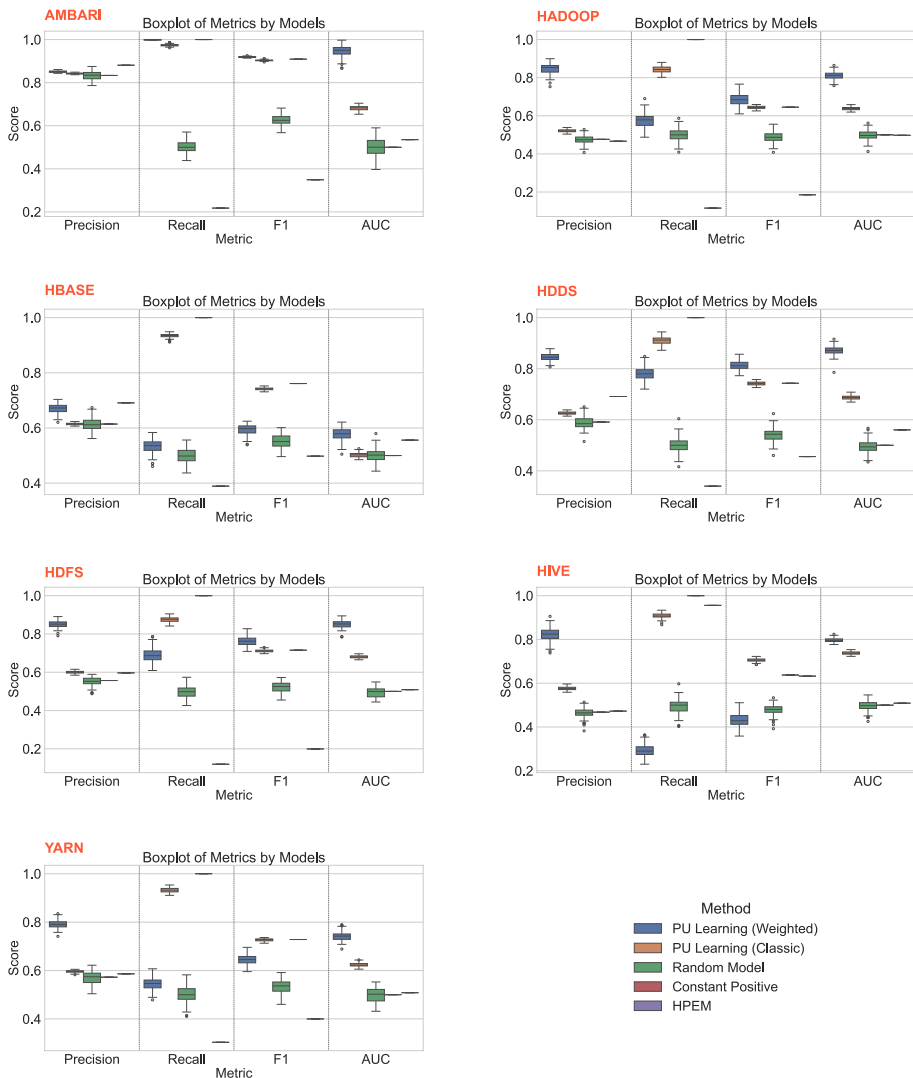


Fig. 9 Performance metrics of the four selected models across the seven studied projects

compared to AMBARI and HADOOP. However, the weighted Elkan & Noto model significantly outperforms the baseline models. The baseline models consistently exhibit lower AUC, Precision, and F1 scores, underscoring the predictive capabilities of our PU Learning models in identifying unrelated build failures. This demonstrates that our models are effective tools for detecting.

Overall, our results suggest that automatically predicting unrelated build failures for developers is feasible. Based on the predictions, developers can obtain additional information, helping their decision-making on whether they should proceed with the merge. An indication of the likelihood that a failure is unrelated can also reduce efforts on investigating the cause for the build breakage, since the build failure is indicated to be unrelated to the current push.

Table 7 Comparison of models across different projects. The first four columns represent the median values of the metrics. The subsequent four columns show the mean values with their standard deviations, which provides insights into the variability of the metrics across our 100 runs

Project	Models	Precision	Recall	F1	AUC	Precision $\pm$ Std	Recall $\pm$ Std	F1 $\pm$ Std	AUC $\pm$ std
AMBARI	PU Learning (Weighted Elkan & Noto)	0.84	1.00	0.91	0.97	0.84 $\pm$ 0.0034	1.00 $\pm$ 0.0014	0.91 $\pm$ 0.0021	0.96 $\pm$ 0.0259
	PU Learning (Classic Elkan & Noto)	0.83	0.97	0.89	0.62	0.83 $\pm$ 0.0025	0.97 $\pm$ 0.0046	0.89 $\pm$ 0.0025	0.62 $\pm$ 0.0097
	Random Model	0.83	0.51	0.63	0.50	0.82 $\pm$ 0.0184	0.50 $\pm$ 0.0281	0.63 $\pm$ 0.0251	0.50 $\pm$ 0.0378
	Constant Positive Model	0.83	1.00	0.90	0.50	0.83 $\pm$ 0.0000	1.00 $\pm$ 0.0000	0.90 $\pm$ 0.0000	0.50 $\pm$ 0.0000
	HPEM Approach	0.88	0.22	0.35	0.54	0.88 $\pm$ 0.0000	0.22 $\pm$ 0.0000	0.35 $\pm$ 0.0000	0.54 $\pm$ 0.0000
HADOOP	PU Learning (Weighted Elkan & Noto)	0.85	0.59	0.70	0.82	0.85 $\pm$ 0.0309	0.59 $\pm$ 0.0524	0.70 $\pm$ 0.0402	0.82 $\pm$ 0.0217
	PU Learning (Classic Elkan & Noto)	0.52	0.84	0.64	0.64	0.52 $\pm$ 0.0066	0.84 $\pm$ 0.0168	0.64 $\pm$ 0.0086	0.64 $\pm$ 0.0063
	Random Model	0.47	0.50	0.48	0.50	0.47 $\pm$ 0.0202	0.50 $\pm$ 0.0344	0.48 $\pm$ 0.0248	0.50 $\pm$ 0.0240
	Constant Positive Model	0.47	1.00	0.64	0.50	0.47 $\pm$ 0.0000	1.00 $\pm$ 0.0000	0.64 $\pm$ 0.0000	0.50 $\pm$ 0.0000
	HPEM Approach	0.47	0.12	0.19	0.50	0.47 $\pm$ 0.0000	0.12 $\pm$ 0.0000	0.19 $\pm$ 0.0000	0.50 $\pm$ 0.0000
HBASE	PU Learning (Weighted Elkan & Noto)	0.70	0.58	0.64	0.63	0.70 $\pm$ 0.0142	0.58 $\pm$ 0.0194	0.63 $\pm$ 0.0143	0.63 $\pm$ 0.0191
	PU Learning (Classic Elkan & Noto)	0.62	0.93	0.74	0.54	0.62 $\pm$ 0.0038	0.93 $\pm$ 0.0084	0.74 $\pm$ 0.0043	0.54 $\pm$ 0.0093
	Random Model	0.62	0.50	0.55	0.50	0.62 $\pm$ 0.0224	0.50 $\pm$ 0.0297	0.55 $\pm$ 0.0249	0.50 $\pm$ 0.0270
	Constant Positive Model	0.61	1.00	0.76	0.50	0.61 $\pm$ 0.0000	1.00 $\pm$ 0.0000	0.76 $\pm$ 0.0000	0.50 $\pm$ 0.0000
	HPEM Approach	0.69	0.39	0.50	0.56	0.69 $\pm$ 0.0000	0.39 $\pm$ 0.0000	0.50 $\pm$ 0.0000	0.56 $\pm$ 0.0000
HDDS	PU Learning (Weighted Elkan & Noto)	0.83	0.80	0.82	0.86	0.83 $\pm$ 0.0184	0.79 $\pm$ 0.0339	0.81 $\pm$ 0.0232	0.86 $\pm$ 0.0321
	PU Learning (Classic Elkan & Noto)	0.61	0.89	0.73	0.69	0.61 $\pm$ 0.0059	0.89 $\pm$ 0.0142	0.73 $\pm$ 0.0074	0.69 $\pm$ 0.0065
	Random Model	0.57	0.50	0.53	0.50	0.57 $\pm$ 0.0262	0.50 $\pm$ 0.0291	0.53 $\pm$ 0.0261	0.50 $\pm$ 0.0318
	Constant Positive Model	0.57	1.00	0.73	0.50	0.57 $\pm$ 0.0000	1.00 $\pm$ 0.0000	0.73 $\pm$ 0.0000	0.50 $\pm$ 0.0000
	HPEM Approach	0.69	0.34	0.46	0.56	0.69 $\pm$ 0.0000	0.34 $\pm$ 0.0000	0.46 $\pm$ 0.0000	0.56 $\pm$ 0.0000

Table 7 (continued)

Project	Models	Precision	Recall	F1	AUC	Preci- sion $\pm$ Std	Recall $\pm$ Std	F1 $\pm$ Std	AUC $\pm$ std
HDFS	PU Learning (Weighted Elkan & Noto)	0.88	0.66	0.75	0.85	0.88 $\pm$ 0.0161	0.66 $\pm$ 0.0405	0.75 $\pm$ 0.0292	0.85 $\pm$ 0.0200
	PU Learning (Classic Elkan & Noto)	0.59	0.87	0.70	0.68	0.59 $\pm$ 0.0062	0.88 $\pm$ 0.0128	0.70 $\pm$ 0.0070	0.68 $\pm$ 0.0087
	Random Model	0.55	0.50	0.53	0.50	0.55 $\pm$ 0.0221	0.50 $\pm$ 0.0293	0.52 $\pm$ 0.0243	0.50 $\pm$ 0.0242
	Constant Positive Model	0.55	1.00	0.71	0.50	0.55 $\pm$ 0.0000	1.00 $\pm$ 0.0000	0.71 $\pm$ 0.0000	0.50 $\pm$ 0.0000
	HPEM Approach	0.60	0.12	0.20	0.51	0.60 $\pm$ 0.0000	0.12 $\pm$ 0.0000	0.20 $\pm$ 0.0000	0.51 $\pm$ 0.0000
HIVE	PU Learning (Weighted Elkan & Noto)	0.82	0.30	0.44	0.78	0.82 $\pm$ 0.0280	0.30 $\pm$ 0.0275	0.43 $\pm$ 0.0312	0.78 $\pm$ 0.0100
	PU Learning (Classic Elkan & Noto)	0.57	0.90	0.70	0.73	0.57 $\pm$ 0.0062	0.90 $\pm$ 0.0111	0.70 $\pm$ 0.0063	0.73 $\pm$ 0.0068
	Random Model	0.47	0.51	0.49	0.50	0.47 $\pm$ 0.0201	0.51 $\pm$ 0.0331	0.49 $\pm$ 0.0245	0.50 $\pm$ 0.0235
	Constant Positive Model	0.46	1.00	0.63	0.50	0.46 $\pm$ 0.0000	1.00 $\pm$ 0.0000	0.63 $\pm$ 0.0000	0.50 $\pm$ 0.0000
	HPEM Approach	0.47	0.96	0.63	0.51	0.47 $\pm$ 0.0000	0.96 $\pm$ 0.0000	0.63 $\pm$ 0.0000	0.51 $\pm$ 0.0000
YARN	PU Learning (Weighted Elkan & Noto)	0.81	0.51	0.63	0.74	0.80 $\pm$ 0.0200	0.51 $\pm$ 0.0311	0.62 $\pm$ 0.0265	0.74 $\pm$ 0.0170
	PU Learning (Classic Elkan & Noto)	0.58	0.92	0.71	0.62	0.58 $\pm$ 0.0042	0.92 $\pm$ 0.0105	0.71 $\pm$ 0.0050	0.62 $\pm$ 0.0084
	Random Model	0.56	0.50	0.53	0.50	0.56 $\pm$ 0.0219	0.50 $\pm$ 0.0285	0.53 $\pm$ 0.0239	0.50 $\pm$ 0.0257
	Constant Positive Model	0.56	1.00	0.72	0.50	0.56 $\pm$ 0.0000	1.00 $\pm$ 0.0000	0.72 $\pm$ 0.0000	0.50 $\pm$ 0.0000
	HPEM Approach	0.59	0.30	0.40	0.51	0.59 $\pm$ 0.0000	0.30 $\pm$ 0.0000	0.40 $\pm$ 0.0000	0.51 $\pm$ 0.0000

4.3 Robustness of Evaluation Results

As described in Section 3, we conducted an additional robustness analysis to verify that our reported results are not biased by the evaluation method. We repeated the 10-fold cross-validation process 30 times, where each repetition uses a different random seed to shuffle the data and produce a distinct fold partitioning. This means the data is re-shuffled before each of the 30 repetitions, resulting in 30 genuinely different fold assignments. For each repeat, we computed metrics using two approaches: (1) the *Aggregated* approach, which combines all fold predictions into one confusion matrix before computing metrics (our original evaluation method), and (2) the *Per-Fold* approach, which computes Precision, Recall, F1, and AUC separately for each of the 10 folds, then averages across folds.

Figure 10 presents boxplots of the per-fold evaluation metrics across all 300 evaluations ($30 \text{ repeats} \times 10 \text{ folds}$) for each project. The distributions confirm that the per-fold metrics exhibit low variability, with compact interquartile ranges across most projects. Across all seven projects, the two evaluation approaches produce nearly identical performance estimates, with differences typically within ± 0.01 – 0.04 . The small standard deviations across 30 repeated splits (typically ± 0.01 – 0.03) indicate that our reported results are stable and not driven by outlier splits or the choice of evaluation method.

4.4 Threshold Sensitivity Analysis

To address concerns about misclassification risks in practical code builds, we conducted a threshold sensitivity analysis examining how the decision threshold affects the False Positive Rate (FP Rate), which is the proportion of related failures incorrectly predicted as unrelated. As shown in Fig. 11, the FP Rate can be controlled through threshold selection. At the default threshold of 0.5, FP rates vary across projects, ranging from 6% (HIVE) to 42% (HBASE), excluding AMBARI. By adjusting the threshold:

At threshold 0.6: 5 out of 7 projects achieve FP Rate < 20%

At threshold 0.7: 6 out of 7 projects achieve FP Rate < 10%

This demonstrates that practitioners can select thresholds based on their risk tolerance, trading off between detection capability (recall) and safety (low FP rate). For safety-critical code builds where developers should not miss related failures, we recommend using higher thresholds (0.6–0.8).

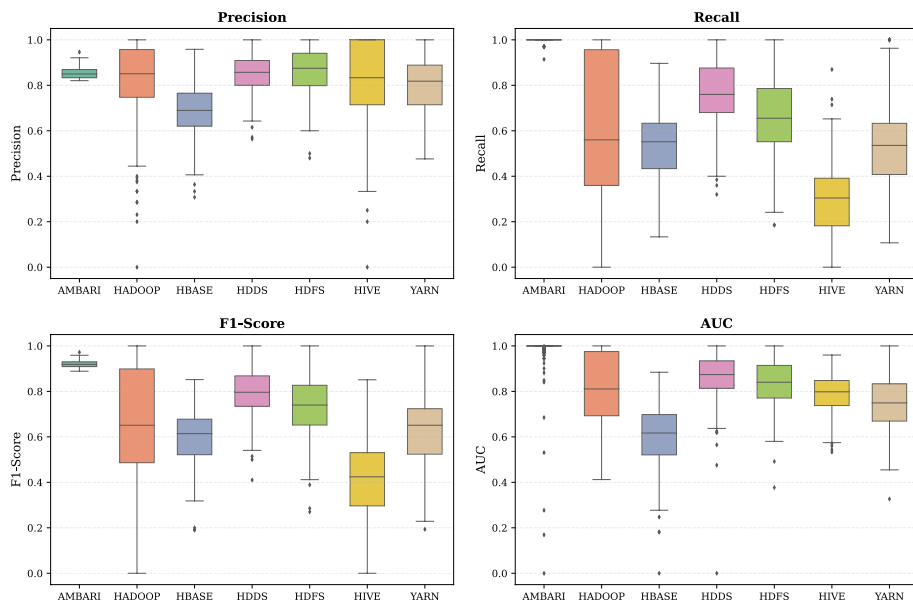


Fig. 10 Distribution of per-fold evaluation metrics across 30 repeated 10-fold cross-validation runs for each project. Each boxplot summarizes 300 per-fold values ($30 \text{ repeats} \times 10 \text{ folds}$)

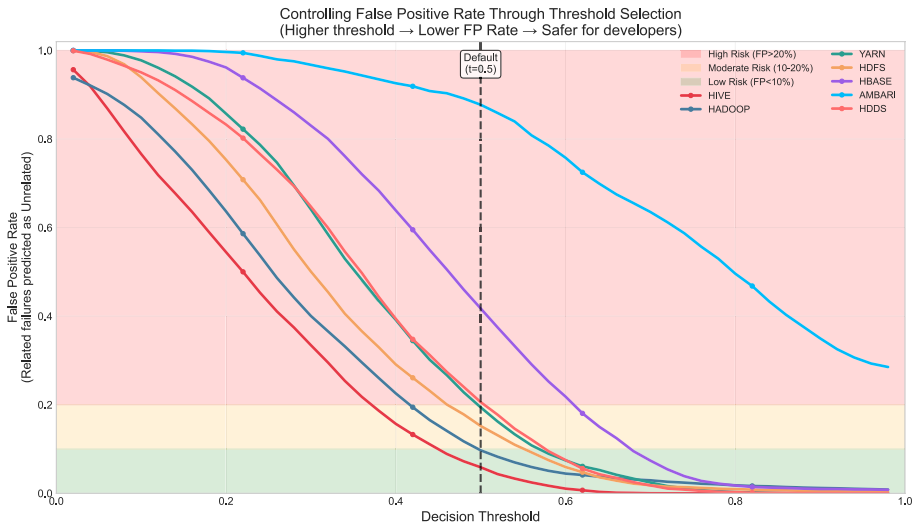


Fig. 11 False Positive Rate at different decision thresholds. The green zone indicates low risk ($FP < 10\%$), yellow indicates moderate risk ($10\text{--}20\%$), and red indicates high risk ($FP > 20\%$). Higher thresholds reduce the risk of incorrectly labeling related failures as unrelated

4.5 Generalization of PU-Learning Models

Table 8 presents the results of cross-project validation, where models trained on data from six projects and tested on the held-out seventh project. Each row reports the performance of our model on the held-out project, with the model trained on the remaining six. We compare the performance of two PU learning models, the Classic Elkan & Noto and Weighted Elkan & Noto, using four evaluation metrics, including precision, recall, F1-score, and AUC. Across all seven projects, the Weighted Elkan & Noto model consistently outperforms the Classic Elkan & Noto model on all four metrics. The weighted variant achieves substantially higher precision, with improvements ranging from 13% to 24%, indicating a stronger ability to avoid false positives (i.e., misclassifying related failures as unrelated). For instance, on the AMBARI project, precision improves from 0.583 to 0.781, and on HDFS, from 0.634 to 0.812. In terms of recall, both models perform strongly, with the weighted model maintaining or improving upon already high recall values (≥ 0.958 across projects except AMBARI). This suggests the model is effective at identifying developer-confirmed unrelated failures. The F1-score, which balances precision and recall, also shows substantial gains with the weighted model (e.g., improving from 0.735 to 0.888 in HBASE and from 0.702 to 0.794 in AMBARI), highlighting the overall robustness of the approach. Similarly, the AUC scores increase significantly with the Weighted Elkan & Noto PU learning, demonstrating stronger discriminative power of the model (e.g., HADOOP improves from 0.684 to 0.962, HDFS from 0.687 to 0.969).

Overall, the results demonstrate that our proposed Weighted Elkan & Noto PU learning model generalizes effectively across different projects. The weighted variant consistently outperforms the classic model in terms of precision, F1, and AUC metrics, establishing itself as a robust and reliable method for identifying unrelated build failures. Furthermore,

Table 8 Cross-project validation results

Test Project	Model	Precision	Recall	F1	AUC
AMBARI	Classic Elkan & Noto	0.583 ± 0.002	0.884 ± 0.004	0.702 ± 0.002	0.647 ± 0.003
	Weighted Elkan & Noto	0.781 ± 0.027	0.809 ± 0.038	0.794 ± 0.027	0.842 ± 0.035
HADOOP	Classic Elkan & Noto	0.641 ± 0.002	0.902 ± 0.005	0.749 ± 0.002	0.684 ± 0.003
	Weighted Elkan & Noto	0.816 ± 0.020	0.967 ± 0.011	0.885 ± 0.013	0.962 ± 0.013
HBASE	Classic Elkan & Noto	0.631 ± 0.003	0.882 ± 0.005	0.735 ± 0.003	0.704 ± 0.003
	Weighted Elkan & Noto	0.819 ± 0.021	0.970 ± 0.010	0.888 ± 0.013	0.969 ± 0.011
HDDS	Classic Elkan & Noto	0.628 ± 0.002	0.892 ± 0.005	0.737 ± 0.003	0.683 ± 0.003
	Weighted Elkan & Noto	0.801 ± 0.020	0.958 ± 0.011	0.873 ± 0.013	0.954 ± 0.014
HDFS	Classic Elkan & Noto	0.634 ± 0.003	0.894 ± 0.005	0.742 ± 0.003	0.687 ± 0.003
	Weighted Elkan & Noto	0.812 ± 0.020	0.971 ± 0.008	0.884 ± 0.012	0.969 ± 0.009
HIVE	Classic Elkan & Noto	0.638 ± 0.003	0.894 ± 0.005	0.745 ± 0.003	0.673 ± 0.003
	Weighted Elkan & Noto	0.810 ± 0.015	0.975 ± 0.007	0.885 ± 0.010	0.968 ± 0.009
YARN	Classic Elkan & Noto	0.626 ± 0.002	0.886 ± 0.005	0.734 ± 0.003	0.684 ± 0.003
	Weighted Elkan & Noto	0.823 ± 0.020	0.961 ± 0.012	0.886 ± 0.012	0.965 ± 0.009

the model maintains strong performance even on projects with limited historical data, highlighting its applicability in low-history environments and its potential for real-world continuous integration systems.

4.6 Importance of Features to Predict Unrelated Build Failures

As shown in Table 9, the features CI latency, Number of Similar Failures and Number of Prior Comments significantly impact the prediction of unrelated CI failures across our study cases.

Table 9 Feature importance results (mean *pm* std values) from PU (weighted) learning

Project Feature	AMBARI	HADOOP	HBASE	HDDS	HDFS	HIVE	YARN
CI Latency	0.3593 ± 0.0021 (Top 1)	0.3711 ± 0.0016 (Top 1)	0.3042 ± 0.0016 (Top 1)	0.2244 ± 0.002 (Top 1)	0.2814 ± 0.0018 (Top 1)	0.1437 ± 0.0015 (Top 3)	0.2342 ± 0.0015 (Top 1)
Is Shared Same Emgs	—	—	—	—	—	0.1514 ± 0.0015 (Top 2)	—
Modified Source Code Files	—	—	—	—	—	—	0.1406 ± 0.0012 (Top 3)
Number of Parallel Issues	0.2895 ± 0.002 (Top 2)	—	—	—	—	—	—
Number of Prior Com- ments	—	0.2095 ± 0.0015 (Top 2)	0.1966 ± 0.0015 (Top 3)	0.1575 ± 0.0014 (Top 3)	0.2173 ± 0.0016 (Top 2)	—	0.172 ± 0.0014 (Top 2)
Number of Similar Fail- ures	0.119 ± 0.0013 (Top 3)	0.0872 ± 0.0011 (Top 3)	0.1986 ± 0.0015 (Top 2)	0.1691 ± 0.0014 (Top 2)	0.1545 ± 0.0013 (Top 3)	0.1892 ± 0.002 (Top 1)	—

CI Latency represents the time elapsed from code patch submission to the push triggering the build. The rationale behind this feature is that if a submitted code patch is quickly pushed, it is more likely that the build failure is related. Conversely, if there is a delay in pushing the patch, the resulting build failure may be unrelated due to other pushes in the meantime. The CI Latency feature is consistently among the top 3 most important features in our seven studied projects.

Number of Similar Failures indicates whether a CI failure shares a similar error message with a past build failure, meaning that if a past failure presents a similar error message to the current failure, it may indicate that the present failure is a repetition of a previous, and unrelated, failure (i.e., the cascading failures discussed by Ghaleb et al. (2019a)). This factor is among the top 3 most important features in HIVE(Top 1), HBASE(Top 2), HDDS(Top 2) AMBARI(Top 3), HADOOP (Top 3) and HDFS(Top 3).

The Number of Prior Comments also emerges as a top 3 important feature in the five projects: HADOOP, HBASE, HDDS, HDFS and YARN. This feature helps gauge the historical discussion around an issue, which can significantly indicate whether a build failure is unrelated to the present push. For instance, if an issue requires extensive discussion, it may suggest complexity and connections to other issues, resulting in build failures that are unrelated to the current push, but related to other issues.

Overall, the important features of our model can help developers reason about whether the build failure is likely related or unrelated to their code push. By regularly monitoring these features and taking proactive actions, teams can iteratively mitigate the negative impact of unrelated failures, enhancing their build processes and software quality.

5 Related Work

In this section, we examine research that has been conducted on Continuous Integration and Continuous Delivery, with a focus on empirical studies. We also consider related work on the problem of flaky tests occurring during the CI/CD process as flaky tests are loosely related to the concept of unrelated build failures.

5.1 Empirical Studies on CI Failures Categorization and Improvement

Continuous integration has become an important part of software development. Continuous integration can improve the code quality and the delivery ability of the engineering team (Fowler and Foemmel 2006; Meyer 2014; Shahin et al. 2017; Arachchi and Perera 2018; Bernardo et al. 2018). Hence, the research on CI has attracted people's attention.

Vassallo et al. (2020) researched CI practices, identified anti-patterns, and developed Bart, CD-Linter, and CI-Odor tools. The empirical studies showed the solutions were effective in promoting principled CI practices and preventing decay over time. CI-Odor was validated and found useful by developers. Rausch et al. (2017) analyzed CI build failures in 14 Java projects, identifying 14 error categories (like test failures, and compile errors) and emphasized the need to address flaky tests and maintain healthy build systems for effective CI.

Because CI/CD processes are essential for software development, researchers are interested in ways to make them better. To increase the reliability and effectiveness of Continuous Integration and Continuous Deployment (CI/CD) procedures, Gallaba et al. (2019) conducted empirical research and analyzed the noise and heterogeneity in CI/CD outcome data, discovered anti-patterns in CI/CD requirements, and raised *Hansel and Gretel* as tools and inference-based build acceleration to quicken CI builds. Shi et al. (2019) compared module- and class-level regression test selection (RTS) techniques in a cloud-based CI environment and developed a hybrid RTS technique. RTS reduced testing time and helped avoid false alarms from flaky tests, providing benefits to developers in the CI environment.

To make the CI process more effective, Elbaum et al. (2014) studied the usage of Automated Static Code Analysis Tools (ASCATs) within Continuous Integration (CI) in projects using Travis CI. They provided an effective and efficient adoption of ASCATs in CI, including excluding unnecessary checks, properly configuring tools according to coding guidelines, and documenting decisions to suppress warnings. Knauss et al. (2015) evaluated a method, which is based on analysis of correlations between test-case failures and source code changes, for selecting a suitable set of functional regression tests on the system level, and found an optimal test suite to execute before the integration. Hassan (Hassan 2019) developed an approach called *HireBuild* that can automatically fix build errors involving build scripts. Reinforcement learning techniques, for example, have proved essential in enhancing the CI process. Yang et al. (2020) conducted a systematic study of the reward function and applied reward strategy in CI testing and sought to improve CI testing efficiency. Similarly, two novel reward functions were proposed by Li et al. (2021), which focused on the impact of failure position in the test case history execution sequence, which is the Average Position Exponential Weight (APEW) reward function and the Average Position Quadratic Weight (APQW) reward function, respectively.

These studies focused on CI build failures of all types, including those caused by code pushes or not. Instead of focusing on detecting general CI failures, our focus is on identifying build failures that are unrelated to the most recent push. Our approach can save developers from efforts on checking whether a build failure is related to their current push. Our work complements previous research, since a development team can benefit from multiple approaches when managing their CI pipelines.

5.2 Studies on the Flaky Tests

In our work, we investigate **legitimate** failures that are unrelated to the current push. However, a related problem is that of flaky tests, which are tests that produce inconsistent results, posing a significant challenge in software testing. There has been an increased focus on the issue of test flakiness in recent years, with a number of studies exploring the causes and effects of flaky tests in both open-source and proprietary software.

Luo et al. (2014)'s research on flaky tests can be traced back to their study of 51 Apache projects, in which they classified the root causes of these tests. The results showed that flaky tests undermine regression testing, caused by race conditions, external dependencies, and stateful tests. This study provided insights and implications that can guide future research on the topic of avoiding flaky tests. Eck et al. (2019) categorized 200 tests and polled 121 developers online to determine how developers perceive flaky tests. They discovered four new flaky test categories, including the effects of flakiness on resource allocation and software reliability, as well as difficulties in recreating and spotting flaky behavior. Ahmad et al. (2021) conducted a survey with developers to understand their perceptions of test flakiness, including how they define flaky tests and the factors that contribute to their presence. The study found that several key factors, including software product quality and the quality of the test suite, are affected by test flakiness. Previous research (Liblit et al. 2005; Palomba and Zaidman 2017; Thorve et al. 2018) revealed that concurrency is the main reason for flaky tests, therefore detecting and repairing concurrency can partially alleviate flaky tests. Silva et al. (2020) proposed SHAKER, a lightweight technology, which is more effective and resource-saving than RERUN for detecting the flaky test caused by concurrency.

Flaky build failures can also be caused by configuration and infrastructure aspects. VERICI, a method by Santolucito et al. (2022) for localizing CI configuration issues at the code level, takes advantage of commit and build history in CI settings to forecast build failures and pinpoint root causes. VERICI was introduced to help improve the CI process and avoid CI failures which are not triggered by code modifications.

Adriaan et al. (2017) identified the flaky tests from Travis and found that 18% of test suite executions fail and that 13% of these failures are flaky. They provided an approach to identify the flaky tests based on the build status history. Doriane et al. (2022) tried to detect flaky tests based on textual similarity to build logs of prior brown builds (a build failure that changes to success on at least one build rerun without changing the build setup or source code) and the model achieved the F-score value at 53% on the studied projects. Johannes et al. (2021) drew attentions on intermittent failures, and proposed classification models using job telemetry data to diagnose failure patterns. Their approach achieves precision scores of 73%.

Unrelated failures are a different problem from flaky tests. Unrelated failures are indeed legitimate and not non-deterministic like flaky tests. However, although certain CI failures are legitimate they may not be related to the push of a certain developer, which is a common scenario in intensive software development projects. The relatedness of a failure to code pushes can impact the decision to merge a change.

5.3 CI Build Outcome Prediction

Build failures in Continuous Integration (CI) systems often occur during the software development process. Previous studies have aimed to develop predictive models to identify when a build is likely to fail in advance and mitigate failures. Jin and Servant (Jin and Servant 2020) introduced a cost-effective approach to CI by predicting build failures and selectively skipping builds that are unlikely to succeed, thus reducing unnecessary resource consumption and accelerating feedback loops. Chen et al. (2020) proposed a history-aware build outcome prediction model that leverages historical build data to predict future build outcomes, aiming to mitigate potential build failures. Saidani et al. (2020b) developed a multi-objective genetic programming approach to predict build failures, addressing the imbalance of build outcomes and enhancing the build process. Machalica et al. (2019) proposed a predictive test selection strategy that selects a subset of tests to perform for each build change, reducing infrastructure costs while maintaining high fault detection rates. Although these studies focus on predicting build failures based on various factors such as historical data, developer changes, and test outcomes, our research specifically targets “unrelated build failures” that occur due to factors not directly related to the recent code changes, such as misconfigurations, overlooked dependencies, or unintended interactions between components. Our approach uncovers unrelated build failure patterns that traditional models miss, and aims to identify and mitigate unrelated failures by analyzing issue reports fields, and their interdependencies, providing a more targeted solution to a specific class of build failures.

Our main goal is to identify build failures that are unrelated to the recent push, which may confuse developers in their decision-making process because they invest time investigating how to fix a failure, which can lead them to perceive that such a failure is not related to their push. Upon discovering that failures are unrelated, developers may choose to integrate their changes if the team assesses that it is safe to do so. Alternatively, they could at least understand the actual scope of the failure and target its root cause without thinking that the failure is related to their current push. As such, our work is substantially different from previous CI build outcome prediction research.

One could argue that unrelated failures would be impossible to occur when proper CI practices are in place, i.e., only merging code upon passing builds. However, in large codebases with multiple developers working concurrently, such as the projects we study and those examined by Ghaleb et al. (2019a), one developer’s change can accidentally impact another’s work, potentially causing future build failures that appear to be unrelated to original contributions. Furthermore, as noted by Felidré et al. (2019), projects employing CI practices do not always adhere to the highest standards, which could lead to increased occurrence of unrelated build failures. Therefore, our research is valuable for projects utilizing CI, as it acknowledges that achieving perfection in adhering to these practices may not always be feasible in real-world scenarios.

6 Discussion

This section explores the implications of our findings for developers and researchers, particularly in making decisions about whether a build failure is attributable to the current code push.

6.1 Implications for Developers

Building upon the results of our study about leveraging the PU learning models into the development workflow can enhance the efficiency of debugging Applying our prediction model by PU learning in the development process can give developers an indication of whether a build failure is related to the current code push, which reduces the time spent on manual investigation by developers and enables a faster resolution for developers on continuous integration build issues. By accurately determining the root cause of build failures, developers can allocate their time and resources more effectively and focus on issues within their control rather than unrelated external factors. Our findings and prediction model offer actionable guidance for developers when a build failure is predicted to be unrelated to recent code changes. We examined the follow-up discussions for all 700 issues in our *P* dataset (confirmed unrelated failures) and found that in 589 cases (84.1%), developers eventually committed their patches after confirming the failures were unrelated through investigation (e.g., manual verification, re-running builds). This high rate suggests that correctly identifying unrelated failures helps developers focus their efforts appropriately. To illustrate, in AMBARI-6965,⁹ a developer submitted a configuration patch and received a CI failure notification. Rather than spending time debugging their own code changes, the developer investigated the failure and determined: “*Unit test failure in Hadoop QA is not related to this patch. Verified unit tests manually.*” This assessment allowed the developer to narrow down the investigation scope and confirm that their changes were correct. Our prediction model aims to provide such early indicators, helping developers quickly identify when a failure may not be caused by their changes, so they can redirect their debugging efforts to more relevant areas, such as checking CI configurations, environmental factors, or other code modules that may be the actual source of the problem. When a failure is predicted as “likely related,” developers should prioritize debugging their current changes by examining stack traces for references to modified files and reproducing failures locally. When receiving a “likely unrelated” prediction, we recommend developers: (1) re-run the build in another controlled environment to confirm whether the failure persists, (2) inspect CI logs and investigate potential causes such as infrastructure issues or recent changes by other contributors, and (3) report unrelated failures that persist to the CI/infrastructure team for systematic monitoring. In either case, developers retain control over the final decision.

While certain individual activities (e.g., inspecting logs) may appear applicable to both outcomes, the key distinction lies in the *investigation direction*. A “likely related” prediction directs an **inward-facing** investigation, starting from the developer’s own code changes and aiming for a local code fix. A “likely unrelated” prediction directs an **outward-facing** investigation, starting by ruling out the developer’s own changes (e.g., re-running the build) and then examining external factors such as CI infrastructure or concurrent changes, with the resolution typically involving escalation rather than code modification. This redirection is empirically grounded: 65.1% of confirmed unrelated failures cannot be resolved by modifying the current push (Section 4), making inward-facing debugging ineffective for those cases.

⁹<https://issues.apache.org/jira/browse/AMBARI-6965>

Developers Should be Aware of the Recurring Patterns of Unrelated Build Failures Developers can use the insights from our PU learning model to proactively address recurring patterns (i.e., our found feature importance of PU learning models in Section 3) of unrelated build failures. For example, the long CI latency, where developers delay the code build after the code was pushed. To address the long CI latency, developers can be advised to trigger the build of the code push as soon as possible, minimizing the occurrence of unrelated build failure warnings. This proactive approach not only reduces unnecessary noise in the CI workflow but also ensures that failure warnings are more relevant and timely to the current code pushes. As a result, the overall stability and reliability of the software development life cycle are enhanced, since the system can more effectively differentiate between valid issues and false alarms, leading to more efficient troubleshooting and quicker resolutions.

Our Model Helps Developers Navigate Real-World CI Challenges in Large-Scale Concurrent Development In large-scale projects where multiple developers work concurrently, maintaining a consistently green build is challenging. As Felidré et al. (2019) observed, many projects do not always adhere to the highest standards of CI practices. Our model helps developers identify whether a failure is likely caused by their own push or by other factors such as concurrent changes from other developers. This guidance can help teams navigate real-world CI challenges and focus their investigation efforts more effectively.

6.2 Implications for Researchers

Researchers Should Pay Closer Attention to Unrelated Failures in Code Pushes Based on our findings in Section 2, which indicate that the median time spent identifying unrelated build failures exceeds 4 hours in the studied projects, we recommend that researchers delve deeper into uncovering insights about failures unrelated to code pushes. Although prior research has focused on flaky tests occurring during the test workflow, our results in Section 4 reveal that it is not sufficient to look at only the flaky test. By studying such unrelated failures, researchers can gain deeper insights into the factors influencing how different build failures impact the efficiency of the software development process.

Researchers Should Explore Broader Applications of PU Learning Across Diverse Software Engineering Domains Our work reveals the potential of PU learning in addressing challenges where labels about whether a build failure is related to current code push are not reliable. Missing labels are a common challenge in real-world applications, particularly when researchers aim to apply machine learning for classification tasks. Our work demonstrates that PU learning can serve as a feasible and promising methodology for addressing other Software Engineering challenges, such as detecting rare bugs, identifying anomalies in log analysis, or uncovering under-documented code segments.

6.3 Replication Study: External Validation Beyond the Apache Ecosystem

To evaluate the generalizability of our approach beyond the Apache ecosystem, we conducted a replication study on a non-Apache project that uses a different CI platform and issue tracking systems (i.e., GitHub).

We employed a systematic project selection process to identify a suitable replication target using the GitHub API. Our selection criteria served two purposes. Study-specific criteria ensured that candidate projects have sufficient CI build data and PR-level metadata for our PU-learning pipeline. We required candidates to use GitHub Actions as their primary CI platform (differing from the Jenkins-based pipelines in our Apache projects), have at least 500 workflow runs, 100 failed builds, and 500 pull requests, and provide publicly accessible build logs. Maturity criteria ensured that candidates represent actively maintained, real-world software projects. Following recommendations from Kalliamvakou et al. (2014) and Munaiah et al. (2017) on using GitHub metrics to identify engineered software projects, we required at least 1,500 stars, 3,000 forks, and 15,000 commits. We excluded forked repositories to avoid duplicates. Applying these criteria to the top 50 Java repositories on GitHub yielded five eligible candidates, namely Keycloak, Apache Kafka, Selenium, Spring Boot, and Spring Framework. We ranked these candidates using a weighted composite score based on PR count (30%), failed build count (40%), and the prevalence of unrelated failure keywords in PR comments (30%). **Keycloak** ranked first and is the only non-Apache project among the top candidates, making it the most suitable replication target for evaluating our approach outside the Apache ecosystem. The full ranking and selection script are available in our replication package¹⁰. **Keycloak** is an open-source identity and access management solution maintained by Red Hat, with over 31,000 stars, nearly 30,000 commits, and more than 22,000 pull requests. It uses GitHub Actions as its primary CI platform with a 9.0% build failure rate, providing sufficient failure data for our analysis.

We collected data from the Keycloak project using the GitHub API and retrieved 1,187 failed builds from GitHub Actions workflow runs across 22,173 pull requests. For each failed build, we collected the associated commit information, including file changes and code churn metrics as described in Section 3. We collected each failed build's PR metadata, including creation time, author, and associated comments with the PR. We also collected build logs for 308 failed builds within GitHub's 90-day log retention period. We followed the same feature extraction logic used in our seven studied Apache projects, adapting it to Keycloak's GitHub-based workflow. We collected the following features, including *Number of Parallel Issues*, *Number of Prior Comments*, *CI Latency*, *Has Code Patch*, *Is Shared Same Error Message*, *Number of Similar Failures*, and code churn metrics (i.e., lines added/deleted/modified in source code and configuration files). Several features from our original Apache studies were not applicable to Keycloak and were therefore excluded, i.e., *Is Cross Project* (JIRA-specific feature for linking issues across projects), *Is Subtask* and *Priority* (GitHub PRs do not have a built-in priority field like JIRA issues). Then, we follow the PU-learning framework, as shown in Fig. 7, and construct P , Q , and N datasets¹¹. We randomly sampled 291 failed builds from the total 1,187 failed builds. This sample size was determined using a 95% confidence level and 5% confidence interval, following the same sampling strategy used in our seven Apache studied projects. We applied the same labeling criteria established during the Document Analysis (DA) process described in Section 3. A failure was labeled as "unrelated" (positive) if developers explicitly indicated that the failure was not caused by the current code push. Following the same dual-coder validation process, two coders independently labeled the 291 sampled build failures for constructing the Q and N datasets, achieving a Cohen's Kappa of

¹⁰https://github.com/ckey/unrelated-build-failures-empirical-studies/blob/main/src/response_exp/github_project_selection.py

¹¹https://github.com/ckey/unrelated-build-failures-empirical-studies/tree/main/src/response_exp/data

0.87, indicating almost perfect agreement (Landis and Koch 1977). This is comparable to the inter-rater reliability observed in our Apache Q dataset labeling (Kappa = 0.91). For the P dataset, the 100 positive samples were independently validated by a second coder, achieving a Cohen's Kappa of 0.98, consistent with our Apache P dataset validation (Kappa = 0.98). All disagreements were resolved through consensus discussion. We obtained the final datasets: P (100 samples), Q (199 samples), and N (92 samples) for our generalization case study.

The results in Table 10 demonstrate that our PU-learning approach achieves good performance on Keycloak, with the Weighted PU Learning model achieving an F1-score of 0.84 ± 0.09 , which falls within the range observed on Apache projects (0.44–0.91). Notably, our PU Learning (Weighted) model outperforms the HPEM baseline by +0.19 (+29.6%) in F1-score, indicating that the learned patterns capture more than simple error message matching. The feature importance analysis in Table 11 reveals that *Has Code Patch*, *CI Latency*, and *Number of Similar Failures* are the most important predictors, with *CI Latency* and *Number of Similar Failures* remaining consistent with the findings in our seven studied Apache projects.

The successful replication on Keycloak provides evidence that our approach generalizes beyond the Apache ecosystem and Jenkins-based CI systems to GitHub-hosted projects using GitHub Actions. The underlying patterns of unrelated build failures, particularly *CI Latency* and *Number of Similar Failures*, remain important predictors across different CI platforms. Despite some platform-specific differences in feature importance rankings, the core feature set can effectively predict unrelated failures in diverse project contexts, achieving comparable F1-scores (0.84 on Keycloak vs. 0.44–0.91 on Apache projects).

7 Threats to Validity

In this section, we discuss the threats to validity of our work.

Construct Validity refers to how our inferences are supported by the evidence.

Table 10 Performance comparison across methods (mean $\pm$ std) in the generalization case study on Keycloak

Method	Precision	Recall	F1-Score	AUC
PU Learning	1.000 $\pm$	0.733 $\pm$	0.840 $\pm$	0.926 $\pm$
(Weighted)	0.000	0.124	0.085	0.054
PU Learning	0.981 $\pm$	0.530 $\pm$	0.682 $\pm$	0.838 $\pm$
(Classic)	0.030	0.097	0.087	0.077
HPEM	1.000 $\pm$	0.480 $\pm$	0.648 $\pm$	0.740 $\pm$
	0.000	0.000	0.000	0.000
Constant Positive	0.617 $\pm$	1.000 $\pm$	0.763 $\pm$	0.500 $\pm$
	0.000	0.000	0.000	0.000
Random Model	0.618 $\pm$	0.501 $\pm$	0.553 $\pm$	0.500 $\pm$
	0.033	0.045	0.038	0.000

Table 11 Top-5 feature importances in the generalization case study on Keycloak

Rank	Feature	Importance
1	Has Code Patch	0.2353
2	CI Latency	0.1832
3	Number of Similar Failures	0.0977
4	Number of Parallel Issues	0.0908
5	Number of Prior Comments	0.0568

Throughout our methodology, there are several stages where we encounter threats to validity. The heuristic-based approach we employ to identify potentially unrelated build failures may introduce errors. To quantify the uncertainty of our heuristic-based estimation, we evaluate its performance on dataset U ($N=2,510$), a statistically representative sample with dual-coder validated labels. Table 12 presents the detailed per-project evaluation results. The keyword-based heuristic achieves a precision of 92.2% (309/335), indicating high reliability when it identifies a failure as unrelated. However, the recall is only 26.2% (309/1,179), meaning the heuristic captures approximately one-quarter of the true unrelated failures. The resulting F1-score is 0.41. These metrics indicate that our reported prevalence of 13.33% (Section 2) should be interpreted as a conservative lower bound, as the true proportion of unrelated failures is likely substantially higher. This limitation directly motivates our adoption of PU learning, which can leverage both confirmed positives and unlabeled instances where many true positives remain hidden. Additionally, the false-positive rate of the heuristic varies across projects (from 0% in YARN to approximately 19% in HADOOP), likely due to differences in developer communication styles. However, since dataset P is manually validated, this variation only affects sampling efficiency and does not introduce noise into model training. Our cross-project performance comparison confirms that models remain robust regardless of heuristic false-positive rates.

Another threat is related to the assumption of “*selected completely at random*” (SCAR) when building our PU learning models. For instance, if the SCAR assumption does not hold, the evaluations performed in our models would likely not represent the actual performance in the latent positive instances hidden in the unlabeled population. However, we mitigate this problem by expanding our developer-confirmed unrelated failures dataset to 700 instances, which is enough to manifest a statistically representative sample of 95% confidence level with 4% margin of error. As such, we are confident that our sample used to build our models represents of the distribution of unrelated build failures and, therefore, the SCAR assumption is valid.

Additionally, the metric we use to measure the delay or time-to-recognition of failures, based on timestamps from issue tracking systems, may not fully capture the actual developer effort or the time actively spent diagnosing the failures. Instead, it reflects the elapsed time recorded in

Table 12 Evaluation of the keyword-based heuristic on dataset U

Project	Sample Size	Manual Positive	Heuristic Positive	False Positive	Precision	Recall	F1
AMBARI	322	246	31	3	0.903	0.114	0.20
HADOOP	368	130	67	13	0.806	0.415	0.55
HBASE	375	191	49	5	0.898	0.230	0.37
HDDS	320	149	34	3	0.912	0.208	0.34
HDFS	376	166	59	1	0.983	0.349	0.52
HIVE	379	127	40	1	0.975	0.307	0.47
YARN	370	170	55	0	1.000	0.324	0.49
Total	2,510	1,179	335	26	0.922	0.262	0.41

Sample Size refers to the number of randomly sampled build failures in dataset U for each project. *Manual Positive* refers to the number of samples labeled as unrelated by dual-coder validation ($Q \subseteq U$). *Heuristic Positive* refers to the number of samples identified as unrelated by the keyword heuristic. *False Positive* refers to cases where the heuristic labeled a failure as “unrelated” but manual inspection disagreed

the repository. This limitation is inherent in empirical software engineering research that relies on archival data from software repositories, and while it may affect the precision of inferences about developer engagement, it does not undermine the overall validity of our findings.

External Validity addresses the extent to which our findings can be applied to different contexts, including other software projects. While our paper studies only 7 projects, our main objective was not to achieve the utmost generalizability. Rather, our main objective was to develop an automated approach for identifying unrelated build failures. For this purpose, we use as case studies projects where Continuous Integration (CI) practices are well-established. Through our approach, we achieved promising results, with models exhibiting AUC values from 0.62 to 0.97 in the studied projects. We believe that our methodology can be extended to different projects, provided the necessary features can be computed. However, adaptations might be necessary for varying project types, such as the calculation of additional metrics.

Additionally, our study is retrospective in nature, analyzing historical CI build failures rather than deploying a real-time prediction tool. While we demonstrate alignment between our model's predictions and developers' actual decision-making behavior in historical cases (with 84.1% of developers eventually committing their patches after confirming failures were unrelated), a controlled user study where developers interact with the prediction tool in their actual workflow would provide stronger evidence of practical utility. Such a study could measure whether predictions lead to reduced debugging time and improved developer satisfaction, which we propose as an important direction for future work.

Internal Validity refers to the degree of confidence or certainty with which our study can establish a causal relationship between the independent variable(s) and the dependent variable(s). We emphasize that our model provides information to narrow the investigation scope, not to replace developer judgment. Root cause analysis remains the developer's responsibility. Our model indicates whether a failure is likely related to the current push, not what the specific root cause is. To mitigate the risk of misclassifying relevant failures as unrelated, we apply a conservative high decision threshold that prioritizes precision. In addition, rather than hiding failures predicted as unrelated, the model highlights them on the CI dashboard along with a confidence score. This design provides developers with additional information to support their debugging process, allowing them to review and override predictions as needed to ensure that no critical issues are overlooked.

Another point is, although certain features are derived from JIRA, the proposed approach can be applied to other CI/CD environments where JIRA is not used. Equivalent features can be obtained from GitHub or other platforms; for example, cross-organization commit patterns or repository dependencies may substitute for "Is Cross Project." Consequently, the approach is generalizable to diverse development environments, including GitHub Actions, CircleCI, and TravisCI.

Also, to reduce the risk of overfitting, we conduct PU-Learning with 10-fold cross-validation for 100 iterations. The time-wise and iterative nature of the process allows for repeated model training, ensuring that the model learns from a variety of data selections, yielding a more reliable and representative outcome.

The use of PU-Learning also helps address the problem with the presence of unbalanced data, where positive instances (unrelated failures) are much rarer than negative instances (related failures).

8 Conclusion

In this research paper, we study unrelated build failures, which are not caused by the pushes that triggered them. We characterize and predict the unrelated build failures of the AMBARI, HBASE, HDFS, HDDS, HADOOP, YARN, and HIVE projects. We observe that unrelated build failures take a median of 4 hours to be identified as such, becoming a burden on developers' time.

Our document analysis on the reasons behind unrelated build failures reveals that **unrelated tests** and **external interference** are among the main reasons why developers consider certain build failures unrelated. To predict whether a build failure is unrelated, we propose the use of a semi-supervised learning approach based on positive and unlabeled instances (PU Learning). Our PU-learning models thus not only achieve AUC values from 0.63 ± 0.02 to 0.97 ± 0.03 but also demonstrate precision between 0.70 ± 0.01 and 0.88 ± 0.01 , recall between 0.30 ± 0.03 and 1.00 ± 0.00 , and F_1 -scores between 0.44 ± 0.03 and 0.91 ± 0.00 , outperforming all baselines and showing that our semi-supervised framework can efficiently detect unrelated build failures. Our prediction model aims to give developers an index of decision-making. For instance, the developers can know the probability of whether a build failure is (un)related to a code push, and then can lower the priority in investigating the issues in code based on the prediction result. Finally, we identify (i) the time taken from a submitted patch to the build-triggering push (CI latency); (ii) build failures sharing similar error messages with recent failures; and (iii) the number of comments preceding the build failure all efficient indicators for identifying unrelated build failures.

Our research shows promising results in the identification of unrelated build failures. Our next step is to apply our approach in industry settings to gather feedback about its efficiency and opportunities for improvements.

Appendix A: Feature Definitions and Calculation Formulas

In this appendix, we provide detailed explanations on the feature engineering process used in our study, including the mathematical formulas for computed features and how each feature is retrieved from fields in the issue reports within JIRA.

Feature Calculation Formulas

The following features require specific calculations:

- **Number of Parallel Issues:** Quantifies the count of issues being worked on simultaneously within a project.

$$\text{NumParallelIssues} = \sum_{n=1}^m (\text{Count}(T_n = T_i)) \quad (4)$$

where m denotes the count of issue reports opened prior to the current issue report i , T_n is the date of the n -th issue report opened, and T_i is the date of the current issue report i opened.

- **Number of Prior Comments:** Represents the number of prior comments associated with a build failure comment.

$$\text{NumPriorComments} = \sum_{j < \text{curr}} \text{count}(C_j) \quad (5)$$

where j iterates through the comments before the current failed CI, curr is the index of the current build failure comment, C_j is the comment at index j , and $\text{count}(C_j)$ is the count of comments by developers for each comment C_j .

- **Is Shared Same Emsg:** A boolean value indicating whether a pair of build failures shares the same error message.

$$\text{IsSharedSameEmsg} = \begin{cases} \text{true} & \text{if } C_{\text{curr}} \in C_{\text{prev}} \\ \text{false} & \text{otherwise} \end{cases} \quad (6)$$

where C_{curr} is the error message from the current build failure and C_{prev} is the set of error messages from previous build failure comments.

- **Number of Similar Failures:** Represents the count of build failures that share failed test/exception classes with the current build.

$$\text{NumSimilarFailures} = \sum_{i=1}^n |F_{\text{curr}} \cap F_{\text{pre}_i}| \quad (7)$$

where F_{pre_i} is the set of failed test/exception classes from the i -th previous build, F_{curr} is the set of failed test/exception classes from the current build, and n is the number of previous builds.

JIRA Issue Report Fields

- **Priority, Is Duplicate, Is Blocker, Is Blocked, Is Regression, Is Dependent, Is Incorporates, Is Required, Is Reference, Is Completes, Is Testing, Is Issue Split, Is Supersedes, Is Cloner, Is Container, Is Parent Feature, and Is Child-Issue** can all be directly retrieved from the corresponding fields in the JIRA issue report. Figure 12 illustrates how features such as **Priority**, **Is Blocker**, and **Is Dependent** are extracted from a sample issue report. Note that not all link-type relationships are visible in an issue's report page—if an issue has no recorded relationship with other issues, those corresponding features are treated as **False** by default.

- **Number of Parallel Issues:** For each issue report, we calculate the number of issues that were opened on the same day, based on the report's creation date, and define this as the number of parallel opened issues (see formula above).
- **Is Cross Projects:** As shown in Fig. 12, this feature is set to **True** when the issue report contains a link to another issue that belongs to a different module or project, as indicated in the "Link" field.
- **Number of Prior Comments:** As shown in Fig. 13, this feature represents the number of comments made before the comment reporting the build failure. For example, in the case of Build Failure 2, there are two comments preceding the failure comment, so the number of prior comments is 2.
- **Is Shared Same Emsg and Number of Similar Failures:** These two features are derived by extracting the failed classes from the failure logs. As illustrated in Fig. 14, the failed classes are parsed from the build failure comments. In this example, both failure comments report the same error message (Emsg), resulting in **Is Shared Same Emsg** being set to **True**, and the **Number of Similar Failures** being equal to 1.
- **CI Latency:** This metric is calculated by taking the difference between the timestamp when the code patch is attached to the JIRA issue and the timestamp when the CI build is triggered. Figure 15 illustrates how these timestamps are retrieved and how the CI latency is computed in the given example.
- **Has Code Patch:** This feature is determined by checking whether the build failure comment includes an attached code patch. Figure 15 shows where the code patch appears in the JIRA issue report.

HBase / HBASE-196
remove ant.jar from lib directory

Details

Type: Bug Priority
 Priority: Major
 Affects Version/s: None
 Component/s: None
 Labels: None

Status: CLOSED
 Resolution: Fixed
 Fix Version/s: None

People

Assignee: Unassigned
 Reporter: Owen O'Malley
 Votes: 0
 Watchers: 1

Dates **Issue creation date**

Created: 16/Aug/07 21:25
 Updated: 02/May/13 02:29
 Resolved: 20/Aug/07 18:01

Description

We don't need the lib/jetty-ext/ant.jar and it causes problems if it disagrees with the version of ant that people are using.

Attachments

Attachment	Size	Created
antpatch.txt	1 kB	20/Aug/07 17:44
HADOOP-1726.patch	0.4 kB	20/Aug/07 17:23

Issue Links

is blocked by HBASE-159 [hbase] If master asks region server to shut down, by-pass return of sh... CLOSED Is Blocked = True

is depended upon by HADOOP-1735 javadoc warnings in trunk: Complaints about missing ant dependency CLOSED Is Cross Project = True

Fig. 12 Example JIRA Issue Report showing how Priority, Is Blocker and Is Depended and Number of Parallel Issues are extracted

The screenshot displays a JIRA thread with three comments. The first comment, from Hadoop QA, details test results for a patch, including failures in javadoc, javac, findbugs, core tests, and contrib tests. It includes links to test reports and console output. The second comment, from Michael Stack, states a failure due to a missing class and a retry. The third comment, also from Michael Stack, says 'Retrying patch.' A red bracket on the right side of the second and third comments is labeled 'Number of Prior Comments = 2'. The first and third comments are highlighted with red boxes and labeled 'Build Failure Comment 1' and 'Build Failure Comment 2' respectively.

Build Failure Comment 1

Hadoop QA added a comment - 06/Jan/08 06:30

-1 overall. Here are the results of testing the latest attachment <http://issues.apache.org/jira/secure/attachment/12372570/getclosestbefore-v3.patch> against trunk revision .

@author +1. The patch does not contain any @author tags.

javadoc -1. The javadoc tool appears to have generated messages.

javac +1. The applied patch does not generate any new compiler warnings.

findbugs -1. The patch appears to cause Findbugs to fail.

core tests -1. The patch failed core unit tests.

contrib tests -1. The patch failed contrib unit tests.

Test results: <http://lucene.zones.apache.org:8080/hudson/job/Hadoop-Patch/1485/testReport/>

Checkstyle results: <http://lucene.zones.apache.org:8080/hudson/job/Hadoop-Patch/1485/artifact/trunk/build/test/checkstyle-errors.html>

Console output: <http://lucene.zones.apache.org:8080/hudson/job/Hadoop-Patch/1485/console>

This message is automatically generated.

Build Failure Comment 2

Hadoop QA added a comment - 06/Jan/08 11:21

-1 overall. Here are the results of testing the latest attachment <http://issues.apache.org/jira/secure/attachment/12372570/getclosestbefore-v3.patch> against trunk revision .

@author +1. The patch does not contain any @author tags.

javadoc +1. The javadoc tool did not generate any warning messages.

javac +1. The applied patch does not generate any new compiler warnings.

Michael Stack added a comment - 06/Jan/08 06:35

Failed because I'd forgotten to add the TruncateCommand class as part of a previous commit (HADOOP-2249). Retry.

Michael Stack added a comment - 06/Jan/08 06:35

Retrying patch.

Number of Prior Comments = 2

Fig. 13 Example of Calculating the Number of Prior Comments

The screenshot shows two build log entries for Ambari. The first entry, for build AMBARI-7685, shows a failure in the unit test 'org.apache.ambari.server.actionmanager.TestActionScheduler'. The second entry, for build AMBARI-7714, also shows a failure in the same unit test. A red dashed box highlights the failed class name in both entries. A red arrow points from the label 'Failed Classes' to the highlighted class names in both log entries.

AMBARI-7685

-1 core tests. The patch failed these unit tests in ambari-server:

org.apache.ambari.server.actionmanager.TestActionScheduler

Test results: <https://builds.apache.org/job/Ambari-trunk-test-patch/160//testReport/>

Console output: <https://builds.apache.org/job/Ambari-trunk-test-patch/160//console>

Failed Classes

AMBARI-7714

-1 core tests. The patch failed these unit tests in ambari-server:

org.apache.ambari.server.actionmanager.TestActionScheduler

Test results: <https://builds.apache.org/job/Ambari-trunk-test-patch/161//testReport/>

Console output: <https://builds.apache.org/job/Ambari-trunk-test-patch/161//console>

Fig. 14 Example of Retrieving the Failed Classes from Build Logs

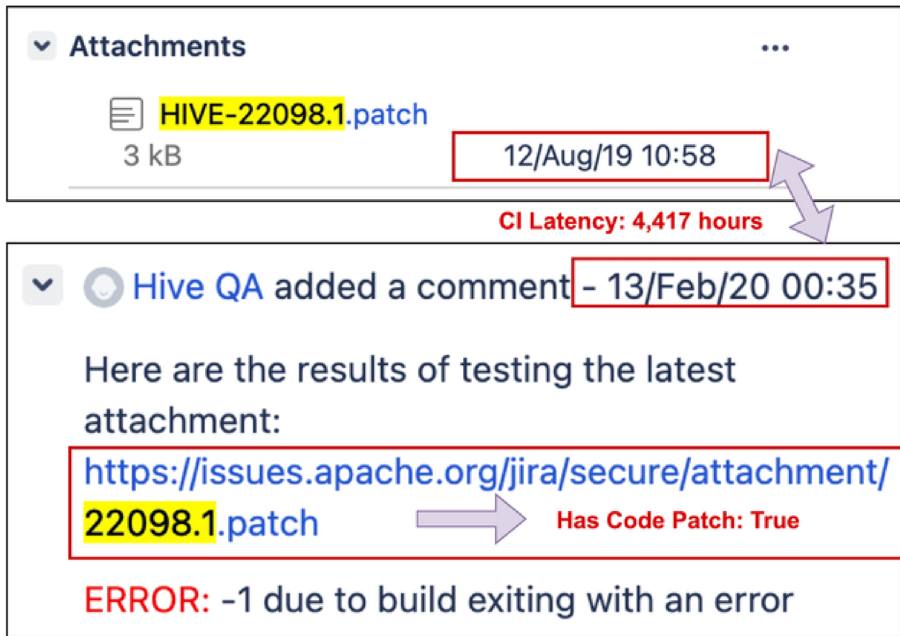


Fig. 15 Example of Retrieving the Has Code Patch and CI Latency

Author Contributions Based on the following abbreviations for the authors: Yonghui (Andie) Huang (Y), Daniel Alencar da Costa (D), Grant Dick (G), and Mariam El Mezouar (M). The contributions of authors based on the CRediT taxonomy are: Conceptualization (D); Data Curation (Y, D); Formal Analysis (Y); Investigation (Y); Methodology (Y, D, G); Writing – Original Draft Preparation (Y); Writing – Review & Editing (D, G, M). All authors read and approved the final manuscript.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions. The research received no external funding.

Data Availability The datasets, scripts, and supporting materials used in this study are available in the replication package at <https://github.com/ckey/unrelated-build-failures-empirical-studies>. Additional supporting materials can be made available upon reasonable request.

Declarations

Ethical Approval Not applicable.

Informed Consent Not applicable.

Conflicts of Interest The authors have no competing interests to declare.

Clinical Trial Number Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are

included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Ahmad A, Leifler O, Sandahl K (2021) Empirical analysis of practitioners' perceptions of test flakiness factors. *Softw Test Verif Reliab* 31(8):e1791
- Ajibode A, Bangash AA, Cogo FR, Adams B, Hassan AE (2025) Towards semantic versioning of open pre-trained language model releases on hugging face. *Empir Softw Eng* 30(3):1–63
- Arachchi S, Perera I (2018) Continuous integration and continuous delivery pipeline automation for agile software project management. In: 2018 Moratuwa Engineering Research Conference (MERCon), IEEE, pp 156–161
- Bell J, Legunsen O, Hilton M, Eloussi L, Yung T, Marinov D (2018) Deflaker: Automatically detecting flaky tests. In: 2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE), IEEE, pp 433–444
- Bernardo JH, da Costa DA, Kulesza U (2018) Studying the impact of adopting continuous integration on the delivery time of pull requests. In: Proceedings of the 15th international conference on mining software repositories, pp 131–141
- Bowen GA (2009) Document analysis as a qualitative research method. *Qualitat Res J*
- Breiman L (2001) Random forests. *Mach Learn* 45(1):5–32
- Chen B, Chen L, Zhang C, Peng X (2020) Buildfast: History-aware build outcome prediction for fast feedback and reduced cost in continuous integration. In: Proceedings of the 35th IEEE/ACM international conference on automated software engineering, pp 42–53
- Çorbacioğlu ŞK, Aksel G (2023) Receiver operating characteristic curve analysis in diagnostic accuracy studies: A guide to interpreting the area under the curve value. *Turkish J Emerg Med* 23(4):195
- Donca I-C, Stan OP, Misaros M, Gota D, Miclea L (2022) Method for continuous integration and deployment using a pipeline generator for agile software projects. *Sensors* 22(12):4637
- Duvall PM, Matyas S, Glover A (2007) Continuous integration: improving software quality and reducing risk. Pearson Educ
- Dybå T, Dingsøyr T (2008) Strength of evidence in systematic reviews in software engineering. In: Proceedings of the second ACM-IEEE international symposium on empirical software engineering and measurement, pp 178–187
- Eck M, Palomba F, Castelluccio M, Bacchelli A (2019) Understanding flaky tests: The developer's perspective. In: Proceedings of the 2019 27th acm joint meeting on european software engineering conference and symposium on the foundations of software engineering, pp 830–840
- Ehsan O, Hassan S, Mezouar ME, Zou Y (2020) An empirical study of developer discussions in the gitter platform. *ACM Trans Softw Eng Methodol (TOSEM)* 30(1):1–39
- El Mezouar M, da Costa DA, German DM, Zou Y (2021) Exploring the use of chatrooms by developers: An empirical study on slack and gitter. *IEEE Trans Software Eng* 48(10):3988–4001
- Elazhary O, Werner C, Li ZS, Lowlind D, Ernst NA, Storey M-A (2021) Uncovering the benefits and challenges of continuous integration practices. *IEEE Trans Software Eng* 48(7):2570–2583
- Elbaum S, Rothermel G, Penix J (2014) Techniques for improving regression testing in continuous integration development environments. In: Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering, pp 235–245
- Elkan C, Noto K (2008) Learning classifiers from only positive and unlabeled data. In: Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining, pp 213–220
- Eloussi L (2015) Determining flaky tests from test failures
- Felidré W, Furtado L, da Costa DA, Cartaxo B, Pinto G (2019) Continuous integration theater. In: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), IEEE, pp 1–10
- Forman G, Scholz M (2010) Apples-to-apples in cross-validation studies: pitfalls in classifier performance measurement. *ACM SIGKDD Explorations Newsl* 12(1):49–57
- Fowler M, Foemmel M (2006) Continuous integration

- Fusilier DH, Montes-y Gómez M, Rosso P, Cabrera RG (2015) Detecting positive and negative deceptive opinions using pu-learning. *Inf Process Manage* 51(4):433–443
- Gallaba K (2019) Improving the robustness and efficiency of continuous integration and deployment. In: 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 619–623
- Ghaleb TA, Da Costa DA, Zou Y (2019a) An empirical study of the long duration of continuous integration builds. *Empir Softw Eng* 24(4):2102–2139
- Ghaleb TA, da Costa DA, Zou Y, Hassan AE (2019b) Studying the impact of noises in build breakage data. *IEEE Trans Software Eng* 47(9):1998–2011
- Ghaleb TA, Hassan S, Zou Y (2022) Studying the interplay between the durations and breakages of continuous integration builds. *IEEE Trans Software Eng* 49(4):2476–2497
- Gousios G, Pinzger M, Deursen Av (2014) An exploratory study of the pull-based software development model. In: Proceedings of the 36th international conference on software engineering, pp 345–355
- Graham H, Owen L (2003) Are there socioeconomic differentials in under-reporting of smoking in pregnancy? *Tob Control* 12(4):434–434
- Hanley JA, McNeil BJ (1982) The meaning and use of the area under a receiver operating characteristic (roc) curve. *Radiology* 143(1):29–36
- Hassan F (2019) Tackling build failures in continuous integration. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, pp 1242–1245
- Herbold S, Trautsch A, Grabowski J (2018) A comparative study to benchmark cross-project defect prediction approaches. In: Proceedings of the 40th international conference on software engineering, pp 1063–1063
- Hertel G, Niedner S, Herrmann S (2003) Motivation of software developers in open source projects: an internet-based survey of contributors to the linux kernel. *Res Policy* 32(7):1159–1177
- Hsieh C-J, Natarajan N, Dhillon I (2015) Pu learning for matrix completion. In: International conference on machine learning, PMLR, pp 2445–2453
- Huang A (2024) unrelated-build-failures-empirical-studies. <https://github.com/ckeys/unrelated-build-failure-s-empirical-studies>. Accessed 26 Nov 2024
- Huang Y, da Costa DA, Zhang F, Zou Y (2019) An empirical study on the issue reports with questions raised during the issue resolving process. *Empir Softw Eng* 24(2):718–750
- Japkowicz N, Shah M (2011) Evaluating learning algorithms: a classification perspective. Cambridge University Press
- Jayasuriya D, Ou S, Hegde S, Terragni V, Dietrich J, Blincoe K (2025) An extended study of syntactic breaking changes in the wild. *Empir Softw Eng* 30(2):1–45
- Jiarpakdee J, Tantithamthavorn C, Treude C (2020) The impact of automated feature selection techniques on the interpretation of defect models. *Empir Softw Eng* 25(5):3590–3638
- Jin X, Servant F (2020) A cost-efficient approach to building in continuous integration. In: Proceedings of the ACM/IEEE 42nd International conference on software engineering, pp 13–25
- Kalliamvakou E, Gousios G, Blincoe K, Singer L, German DM, Damian D (2014) The promises and perils of mining GitHub. In: Proceedings of the 11th working conference on mining software repositories, pp 92–101
- Kitchenham B, Brereton P (2013) A systematic review of systematic review process research in software engineering. *Inf Softw Technol* 55(12):2049–2075
- Knauss E, Staron M, Meding W, Söder O, Nilsson A, Castell M (2015) Supporting continuous integration by code-churn based test selection. In: 2015 IEEE/ACM 2nd International workshop on rapid continuous software engineering, IEEE, pp 19–25
- Labuschagne A, Inozemtseva L, Holmes R (2017) Measuring the cost of regression testing in practice: A study of java projects using continuous integration. In: Proceedings of the 2017 11th joint meeting on foundations of software engineering, pp 821–830
- Lampel J, Just S, Apel S, Zeller A (2021) When life gives you oranges: detecting and diagnosing intermittent job failures at mozilla. In: Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering, pp 1381–1392
- Landis JR, Koch GG (1977) The measurement of observer agreement for categorical data. *Biometrics* 33(1):159–174
- Lanubile F, Ebert C, Prikladnicki R, Vizcaino A (2010) Collaboration tools for global software engineering. *IEEE Softw* 27(2):52
- Li X, Liu B (2003) Learning to classify texts using positive and unlabeled data. In: IJCAI, vol 3, Citeseer, pp 587–592

- Li W, Guo Q, Elkan C (2010) A positive and unlabeled learning algorithm for one-class classification of remote-sensing data. *IEEE Trans Geosci Remote Sens* 49(2):717–725
- Li G, Yang Y, Wu Z, Cao T, Liu Y, Li Z (2021) Weighted reward for reinforcement learning based test case prioritization in continuous integration testing. In: 2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC), IEEE, pp 980–985
- Liblit B, Naik M, Zheng AX, Aiken A, Jordan MI (2005) Scalable statistical bug isolation. *Acm Sigplan Notices* 40(6):15–26
- Liu B, Lee WS, Yu PS, Li X (2002) Partially supervised classification of text documents. *ICML*, vol 2. NSW, Sydney, pp 387–394
- Lüders CM, Bouraffa A, Maalej W (2022a) Beyond duplicates: Towards understanding and predicting link types in issue tracking systems. In: Proceedings of the 19th international conference on mining software repositories, pp 48–60
- Lüders CM, Pietz T, Maalej W (2022b) Automated detection of typed links in issue trackers. In: 2022 IEEE 30th International Requirements Engineering Conference (RE), IEEE, pp 26–38
- Luo Q, Hariri F, Eloussi L, Marinov D (2014) An empirical analysis of flaky tests. In: Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering, pp 643–653
- Machalica M, Samylnin A, Porth M, Chandra S (2019) Predictive test selection. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, pp 91–100
- McIntosh S, Kamei Y, Adams B, Hassan AE (2016) An empirical study of the impact of modern code review practices on software quality. *Empir Softw Eng* 21(5):2146–2189
- McIntosh S, Kamei Y (2018) Are fix-inducing changes a moving target? a longitudinal case study of just-in-time defect prediction. In: Proceedings of the 40th international conference on software engineering, pp 560–560
- Meyer M (2014) Continuous integration and its tools. *IEEE Softw* 31(3):14–16
- Munaiah N, Kroh S, Cabrey C, Nagappan M (2017) Curating GitHub for engineered software projects. *Empir Softw Eng* 22(6):3219–3253
- Nguyen MN, Li X-L, Ng S-K (2011) Positive unlabeled learning for time series classification. In: Twenty-second international joint conference on artificial intelligence, Citeseer
- Olewicki D, Nayrolles M, Adams B (2022) Towards language-independent brown build detection. In: Proceedings of the 44th international conference on software engineering, pp 2177–2188
- Palomba F, Zaidman A (2017) Notice of retraction: Does refactoring of test smells induce fixing flaky tests? In: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 1–12
- Pan C, Pradel M (2021) Continuous test suite failure prediction. In: Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis, pp 553–565
- Perry DE, Siy HP, Votta LG (2001) Parallel changes in large-scale software development: an observational case study. *ACM Trans Softw Eng Methodol (TOSEM)* 10(3):308–337
- Rajbahadur GK, Wang S, Kamei Y, Hassan AE (2017) The impact of using regression models to build defect classifiers. In: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), IEEE, pp 135–145
- Rausch T, Hummer W, Leitner P, Schulte S (2017) An empirical analysis of build failures in the continuous integration workflows of java-based open-source software. In: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), IEEE, pp 345–355
- Saidani I, Ouni A, Chouchen M, Mkaouer MW (2020a) Predicting continuous integration build failures using evolutionary search. *Inf Softw Technol* 128:106392
- Saidani I, Ouni A, Chouchen M, Mkaouer MW (2020b) On the prediction of continuous integration build failures using search-based software engineering. In: Proceedings of the 2020 genetic and evolutionary computation conference companion, pp 313–314
- Santolucito M, Zhang J, Zhai E, Cito J, Piskac R (2022) Learning ci configuration correctness for early build feedback. 2022 IEEE International Conference on Software Analysis. Evolution and Reengineering (SANER), IEEE, pp 1006–1017
- Santos J, Alencar da Costa D, Kulesza U (2022) Investigating the impact of continuous integration practices on the productivity and quality of open-source projects. In: Proceedings of the 16th ACM/IEEE international symposium on empirical software engineering and measurement, pp 137–147
- Santos J, da Costa DA, McIntosh S, Kulesza U (2025) On the need to monitor continuous integration practices. *Empir Softw Eng* 30(5):125

- Shahin M, Babar MA, Zhu L (2017) Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices. *IEEE Access* 5:3909–3943
- Shi A, Zhao P, Marinov D (2019) Understanding and improving regression test selection in continuous integration. In: 2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE), IEEE, pp 228–238
- Silva D, Teixeira L, d'Amorim M (2020) Shake it! detecting flaky tests caused by concurrency with shaker. In: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 301–311
- Soares E, Sizilio G, Santos J, da Costa DA, Kulesza U (2022) The effects of continuous integration on software development: a systematic literature review. *Empir Softw Eng* 27(3):78
- Tantithamthavorn C, Hassan AE, Matsumoto K (2018) The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. *IEEE Trans Software Eng* 46(11):1200–1219
- Teisseyre P, Furmańczyk K, Mielniczuk J (2024) Verifying the selected completely at random assumption in positive-unlabeled learning. *arXiv preprint arXiv:2404.00145*
- Terragni V, Salza P, Ferrucci F (2020) A container-based infrastructure for fuzzy-driven root causing of flaky tests. In: 2020 IEEE/ACM 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), IEEE, pp 69–72
- Thorve S, Sreshtha C, Meng N (2018) An empirical study of flaky tests in android apps. In: 2018 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 534–538
- Turhan B, Menzies T, Bener AB, Di Stefano J (2009) On the relative value of cross-company and within-company data for defect prediction. *Empir Softw Eng* 14(5):540–578
- Vandekerckhove J, Matzke D, Wagenmakers E-J (2015) 14 model comparison and the principle of parsimony. *The Oxford handbook of computational and mathematical psychology*, 300
- Vasilescu B, Yu Y, Wang H, Devanbu P, Filkov V (2015) Quality and productivity outcomes relating to continuous integration in github. In: Proceedings of the 2015 10th joint meeting on foundations of software engineering, pp 805–816
- Vassallo C (2020) Principle-driven continuous integration: simplifying failure discovery and raising anti-pattern awareness. PhD thesis, University of Zurich
- Yang P, Li X-L, Mei J-P, Kwok C-K, Ng S-K (2012) Positive-unlabeled learning for disease gene identification. *Bioinformatics* 28(20):2640–2647
- Yang Y, Li Z, He L, Zhao R (2020) A systematic study of reward for reinforcement learning based continuous integration testing. *J Syst Softw* 170:110787
- Yue Y, De Ridder D, Manning P, Deng JD (2024a) Variational autoencoder learns better feature representations for eeg-based obesity classification. In: International conference on pattern recognition, Springer, pp 179–191
- Yue Y, Deng JD, Chakraborti T, De Ridder D, Manning P (2024b) Unsupervised hybrid deep feature encoder for robust feature learning from resting-state eeg data. In: 2024 46th Annual international conference of the IEEE Engineering in Medicine and Biology Society (EMBC), IEEE, pp 1–5
- Yu H, Han J, Chang KC-C (2002) Pebl: positive example based learning for web page classification using svm. In: Proceedings of the eighth ACM SIGKDD international conference on knowledge discovery and data mining, pp 239–248
- Zampetti F, Bavota G, Canfora G, Di Penta M (2019) A study on the interplay between pull request review and continuous integration builds. 2019 IEEE 26th International conference on Software Analysis. Evolution And Reengineering (SANER), IEEE, pp 38–48
- Zampetti F, Vassallo C, Panichella S, Canfora G, Gall H, Di Penta M (2020) An empirical characterization of bad practices in continuous integration. *Empir Softw Eng* 25:1095–1135
- Zimmermann T, Nagappan N, Gall H, Giger E, Murphy B (2009) Cross-project defect prediction: a large scale experiment on data vs. domain vs. process. In: Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering, pp 91–100
- Zina O et al (2021) The essential guide to doing your research project. Sage

Authors and Affiliations

Yonghui Andie Huang^{1,2}  · Daniel Alencar da Costa¹ · Grant Dick¹ · Mariam El Mezouar³ · Liwen Xiao¹

✉ Yonghui Andie Huang
andie.huang@postgrad.otago.ac.nz; andie.huang@massey.ac.nz

Daniel Alencar da Costa
danielcalencar@otago.ac.nz

Grant Dick
grant.dick@otago.ac.nz

Mariam El Mezouar
mariam.el-mezouar@rmc.ca

Liwen Xiao
xiali449@student.otago.ac.nz

¹ School of Computing, University of Otago, Dunedin, New Zealand

² School of Mathematical and Computational Sciences, Massey University, Palmerston North, New Zealand

³ Department of Mathematics and Computer Science, Royal Military College of Canada, Kingston, Canada